The vulnerable web application was scanned using **OWASP ZAP**, and multiple vulnerabilities were identified in **OWASP Juice Shop**.

Experiment: 15 Analysing Android App Permissions and Mobile Traffic

1. Aim

To analyse Android application network traffic and permissions by intercepting HTTP and HTTPS requests using an Android Emulator and Burp Suite.

2. Objective

- To create and run an Android Virtual Device (AVD).
- To configure a proxy between the Android Emulator and Burp Suite.
- To capture HTTP traffic from the Android device.
- To intercept and analyse HTTPS traffic by installing the Burp Suite CA certificate.

3. Required Tools / Software

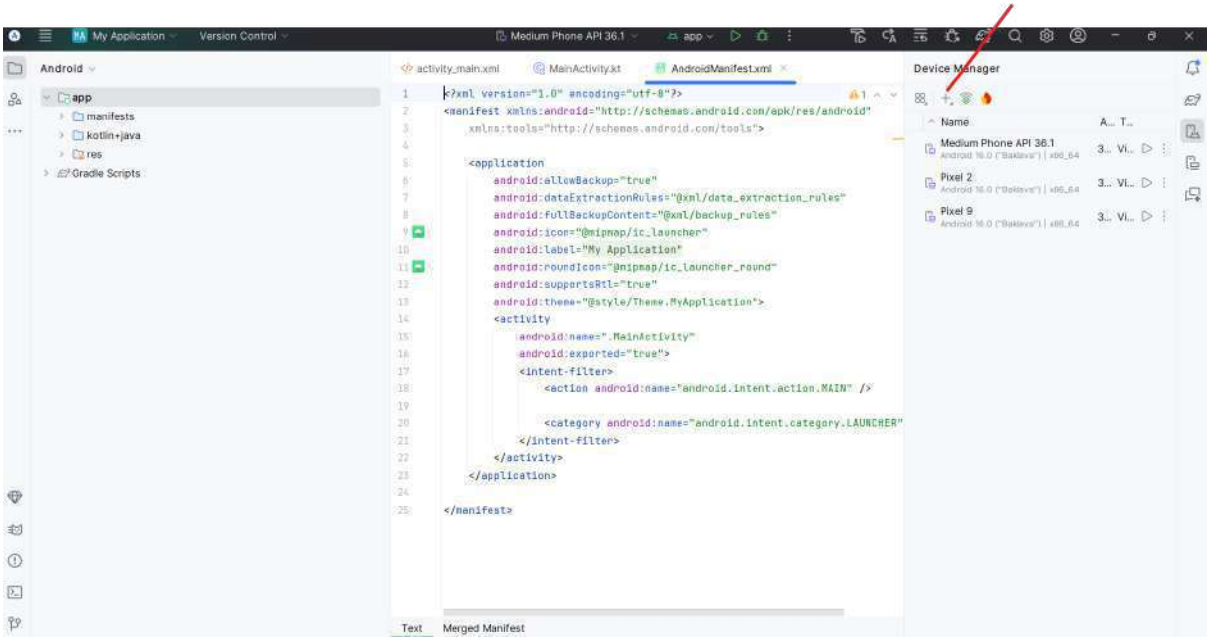
- Android Studio
- Android Emulator (AVD Manager)
- Burp Suite
- ADB (Android Debug Bridge)
- Google Chrome (inside emulator)
- Internet connection

4. Procedure

Step 1: Create Android Virtual Device

1. Install **Android Studio**.
2. Open **AVD Manager**.
3. Create a new Virtual Device.

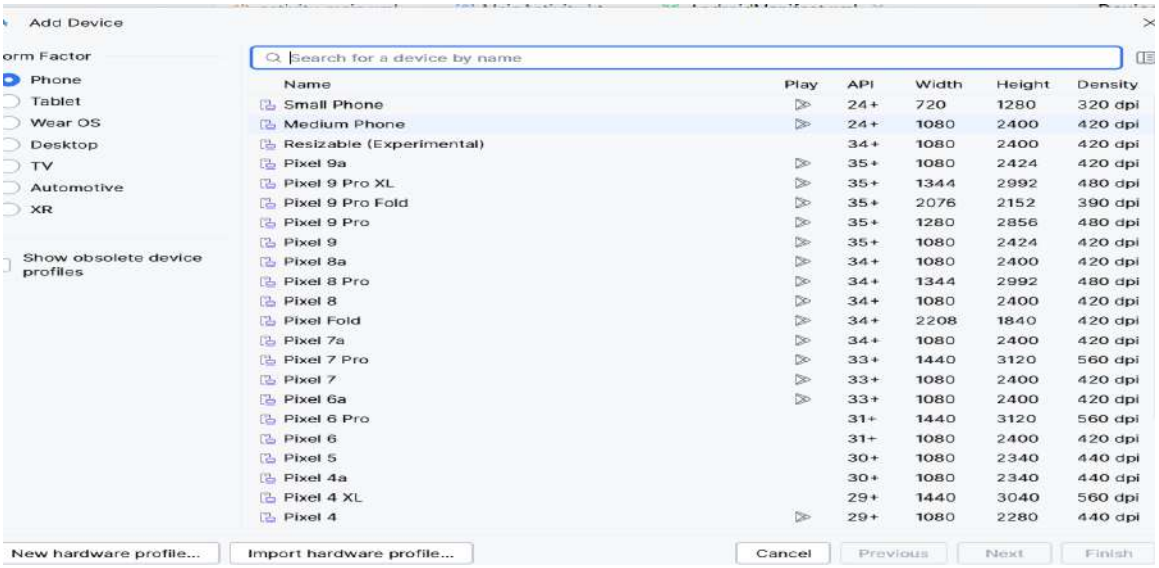
Step-1: Download Android Studio->Create Virtual Device



Step 2: Select Device

Choose any device such as:

- Pixel 2
- Pixel 9



Step-3: Start AVD from Command Prompt1

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\emulator

C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -list-avds

Medium_Phone_API_36.1

Pixel_2

Pixel_9

C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -avd Pixel_9 -http-proxy http://10.0.2.2:8080

Step-4: Setup Proxy:

In AVD

Settings-> Network and Internet-> internet->select icon -> top right corner there is edit->advanced options -> proxy->Change proxy None->Manual->Hostname as 10.0.2.2 ->save

Note:if save option is not working then follow the steps in command prompt

Command Prompt2:

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices

List of devices attached

emulator-5554 device

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080

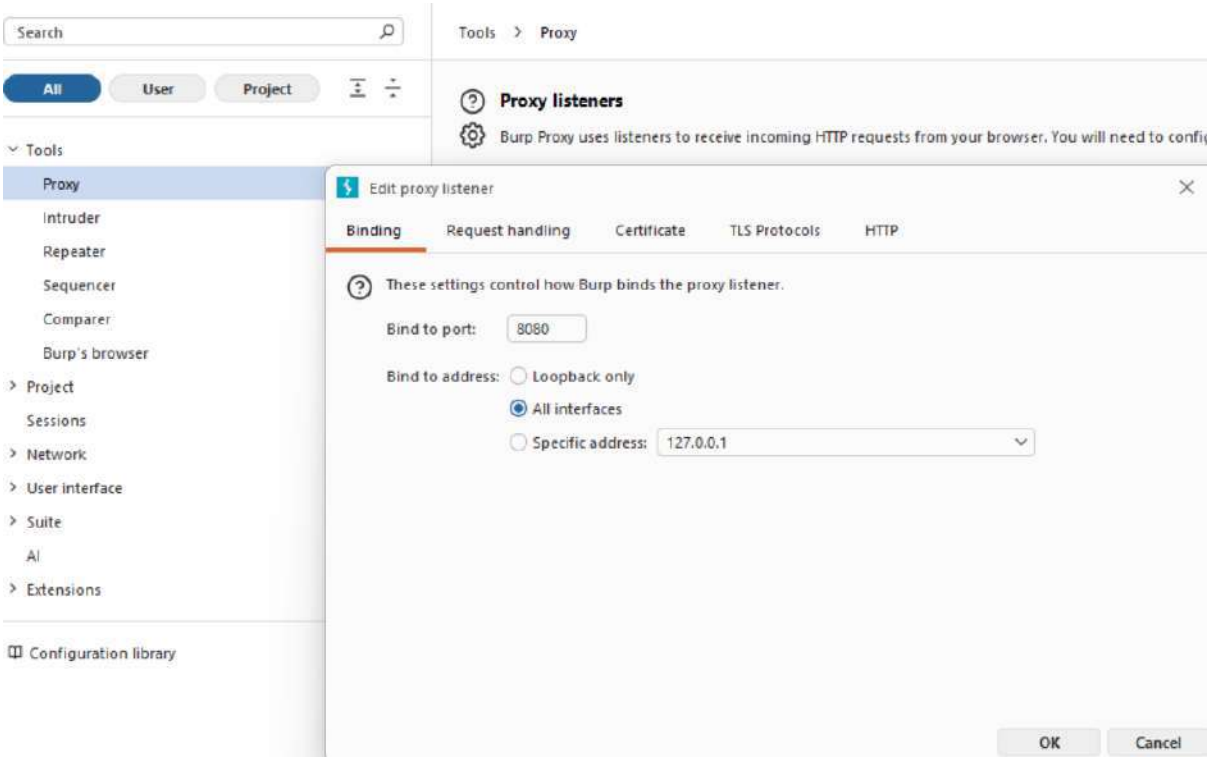
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy

10.0.2.2:8080

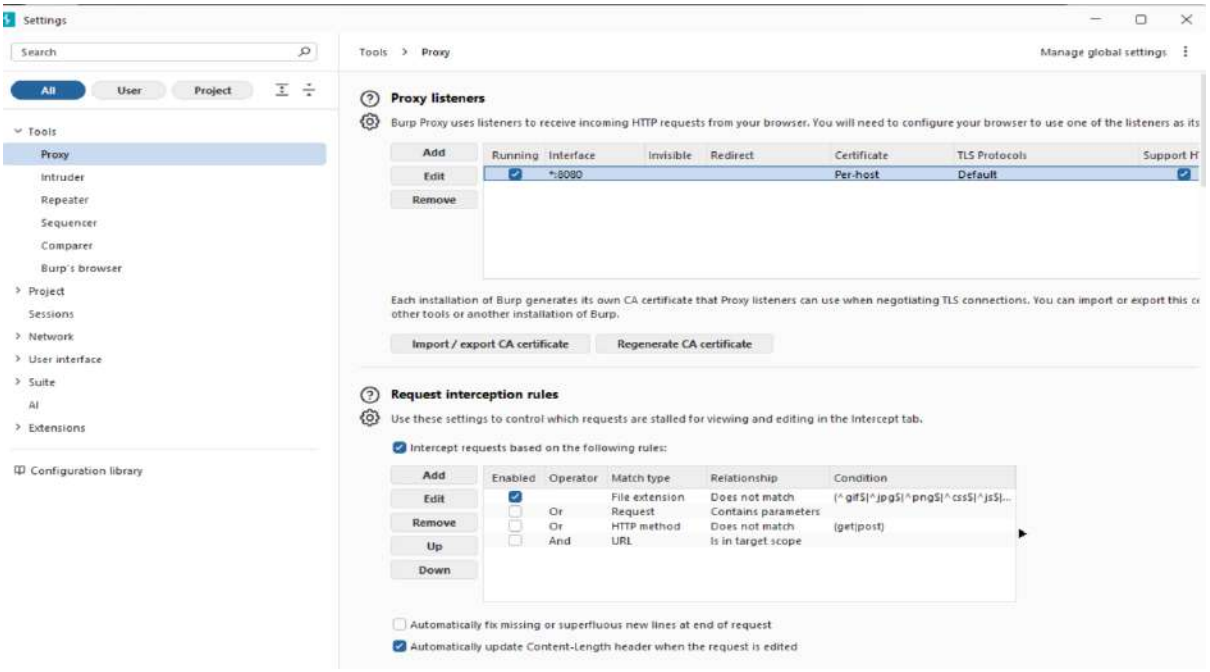
```
C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices
List of devices attached
emulator-5554   device


C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy
10.0.2.2:8080

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>
```

Step 5: In BurpSuit

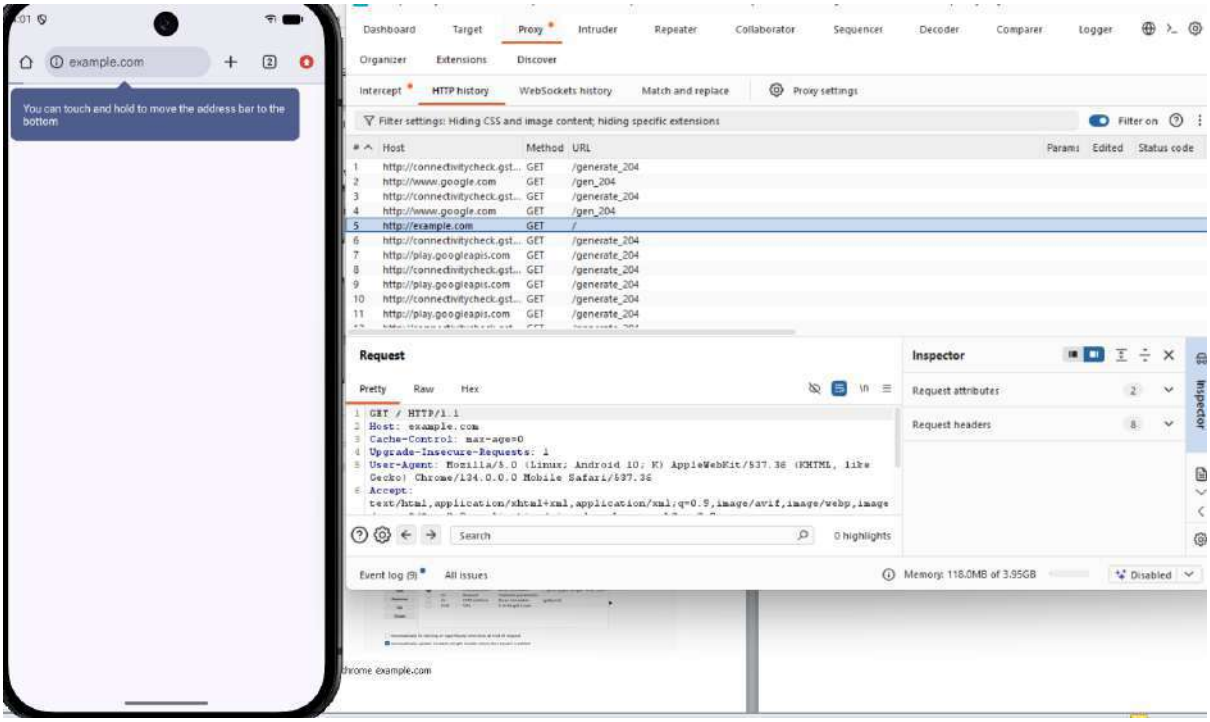
Go procy->options/proxy settings->under Proxy Listeners->Edit->select All interfaces->ok
Go proxy->intercept on



You can see BurpSuit is accepting all interfaces traffic

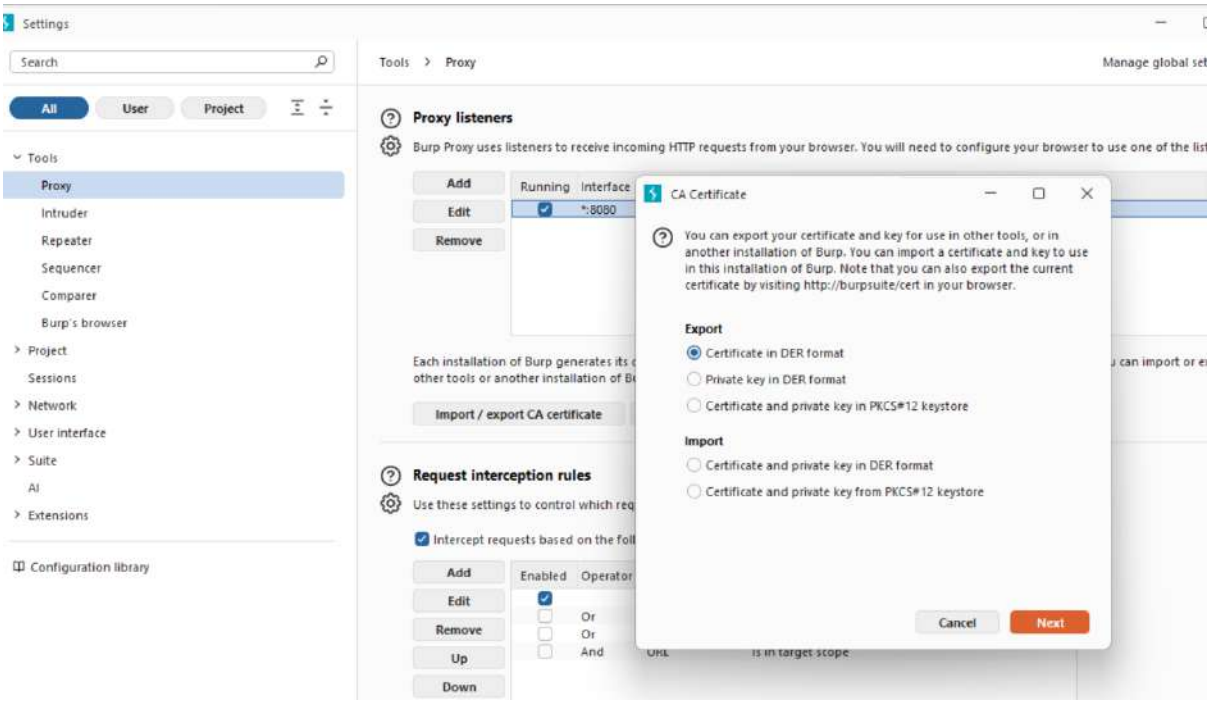


Step 6: in AVD type in chrome example.com and you can see burpsuit HTTP history



Step 7: in Step 5 we only able to http request and response only but if you want to analyse https request and response follow the Procedure to download CA certificate from BunrpSuit.

Go procy->options/proxy settings->import/export CA certificate->Certificate in DER format->select file-> Downloads->give name as burpcer.der



Step 8: Sending CA certificate to AVD

Go to command Prompt2

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/
C:\Users\Admin\Downloads\burpcer.cer: 1 file pushed, 0 skipped. 0.0 MB/s (939 bytes in 0.022s)

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>
```

To install Certificate in AVD

Settings

→ Security & privacy

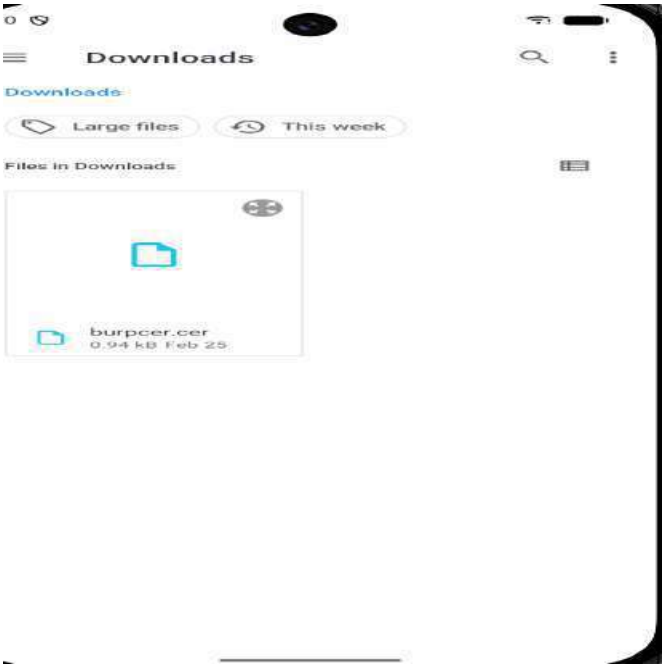
→ More security & privacy

→ Encryption & credentials

→ Install a certificate

→ CA certificate

In AVD we can see burpcer.der certificate



Step 9:verify HTTPS Interception

1. Open browser in emulator.
2. Visit any HTTPS website.

Example:
https://example.com or https://wikipedia.com

3. In Burp Suite, check **Proxy → HTTP History**.

Now HTTPS requests and responses will be visible.

5. Result

Android application network traffic was successfully intercepted and analysed using Burp Suite by configuring proxy settings in the Android emulator and installing the Burp Suite CA certificate.

6. Precautions

- Ensure Burp Suite proxy port (8080) is correctly configured.
- Verify the emulator is running before executing ADB commands.
- Install the CA certificate properly to capture HTTPS traffic.
- Disable intercept when not required to allow traffic to pass normally.

Experiment 16: Testing IoT Device Security (Default Passwords & Open Ports)

Aim

To analyze security weaknesses in a simulated IoT device by identifying:

- Open ports
- Running services
- Service version disclosure
- Default/weak authentication
- Unencrypted communication

Tools Used

- Windows 10 (Host System)
- Docker Desktop
- OWASP Juice Shop
- Kali Linux
- Nmap

Experimental Setup

- A vulnerable web application (Juice Shop) was deployed using Docker on Windows.
- Kali Linux was used as the attacker machine.
- The Windows system IP address was scanned from Kali Linux to simulate IoT network testing.

Attack Scenario Description

In this experiment:

- Kali Linux acted as the attacker machine.
- Windows system hosting the Docker container acted as the IoT device.
- The vulnerable web application (Juice Shop) simulated the IoT dashboard.
- Network scanning was performed from Kali to the Windows IP address over LAN.

This simulates a real-world scenario where an attacker scans an IoT device connected to the same network.

Procedure

Step 1: Deploy Vulnerable IoT Simulation

The following command was used to run Juice Shop in Docker:
docker run -d -p 8090:3000 --name juiceshop bkimminich/juice-shop

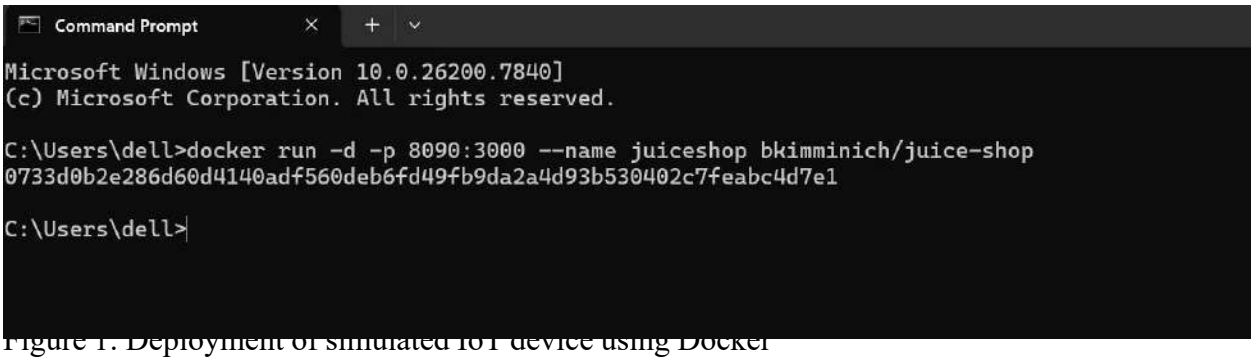


Figure 1: Deployment of simulated IoT device using Docker

The application was accessed at:

http://localhost:8090

Step 2: Identify Host IP Address

On Windows: ipconfig

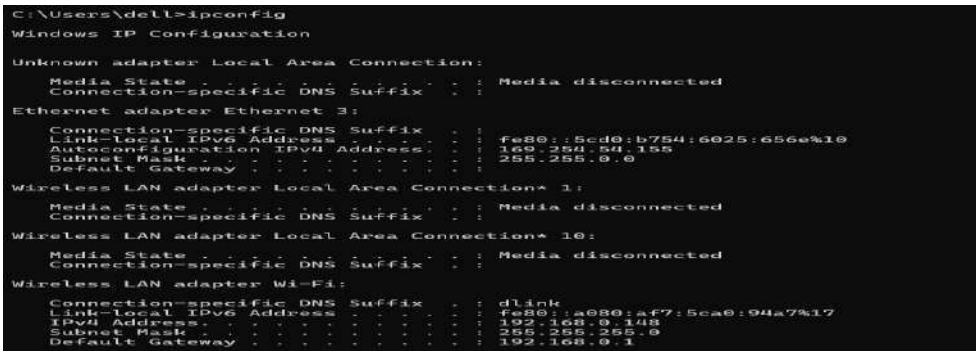


Figure 2: Host system IP address identification

Step 3: Perform Network Scan from Kali Linux

From Kali Linux terminal:

nmap -sV 192.168.0.148

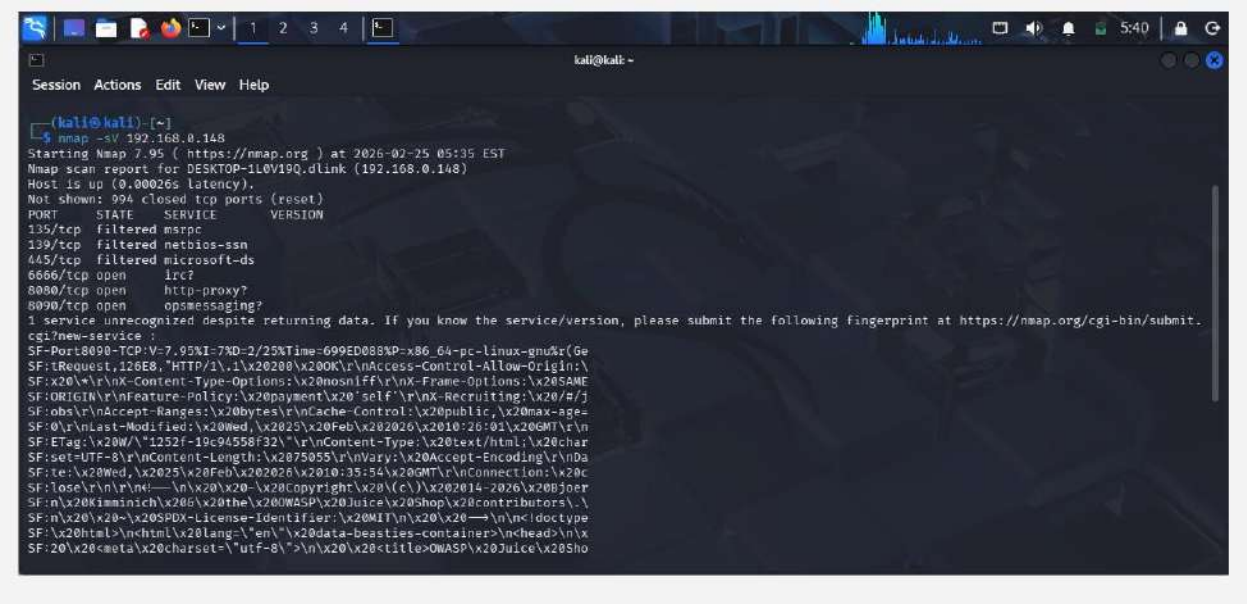
This scan performed:

- Port scanning

Figure 3: Open ports and service enumeration using Nmap

Step 4: Nmap Version Scan (-sV)

- nmap -sV 192.168.x.x output
- Showing service fingerprint / version detection



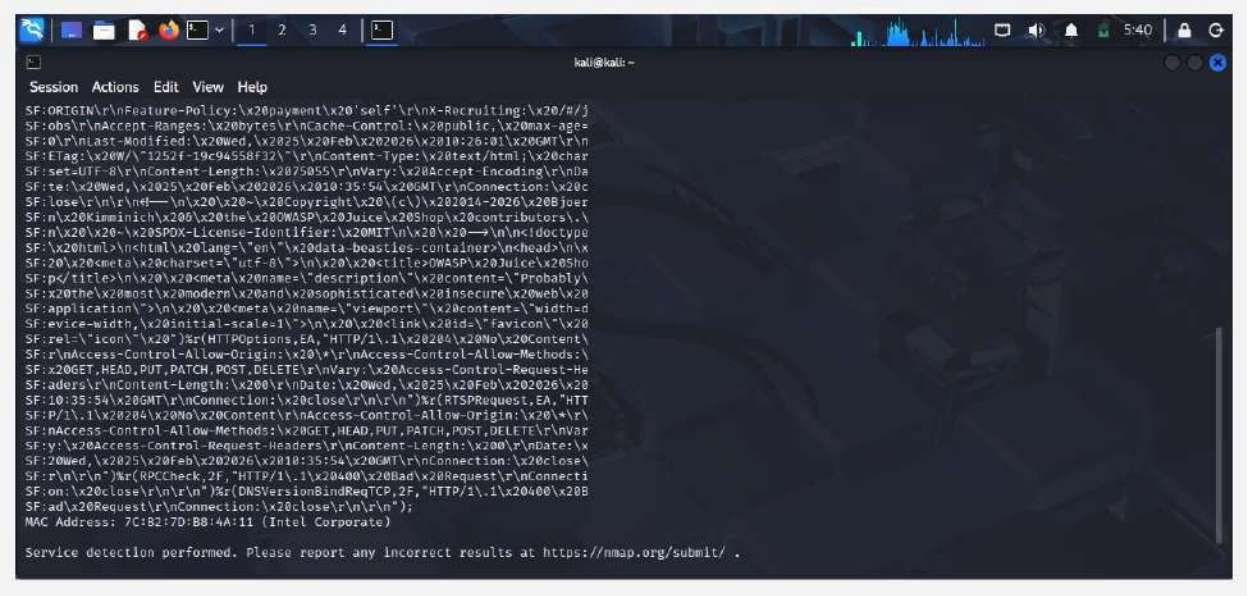


Figure 4: Service enumeration and version detection

Step 5: Accessing the IoT Device Web Dashboard

The simulated IoT device dashboard was accessed through the browser using: http://localhost:8090

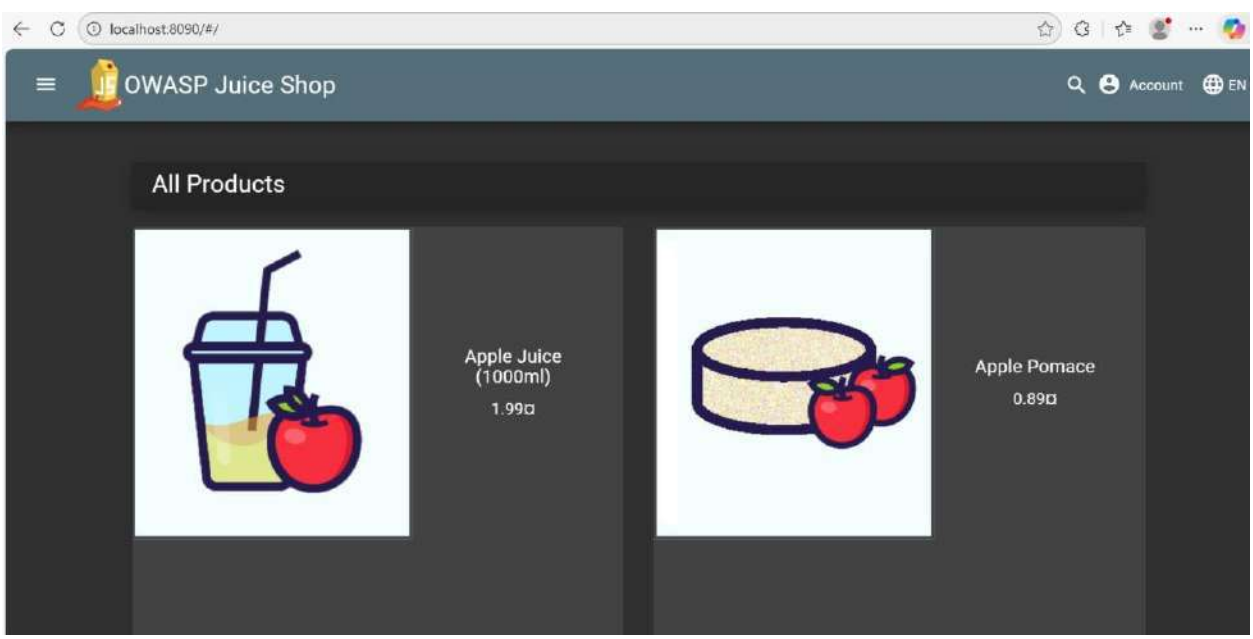


Figure 5: Simulated IoT device web dashboard running on port 8090

Step 6: Testing Default/Weak Credentials

The login functionality was tested using administrator credentials to check for default password vulnerability

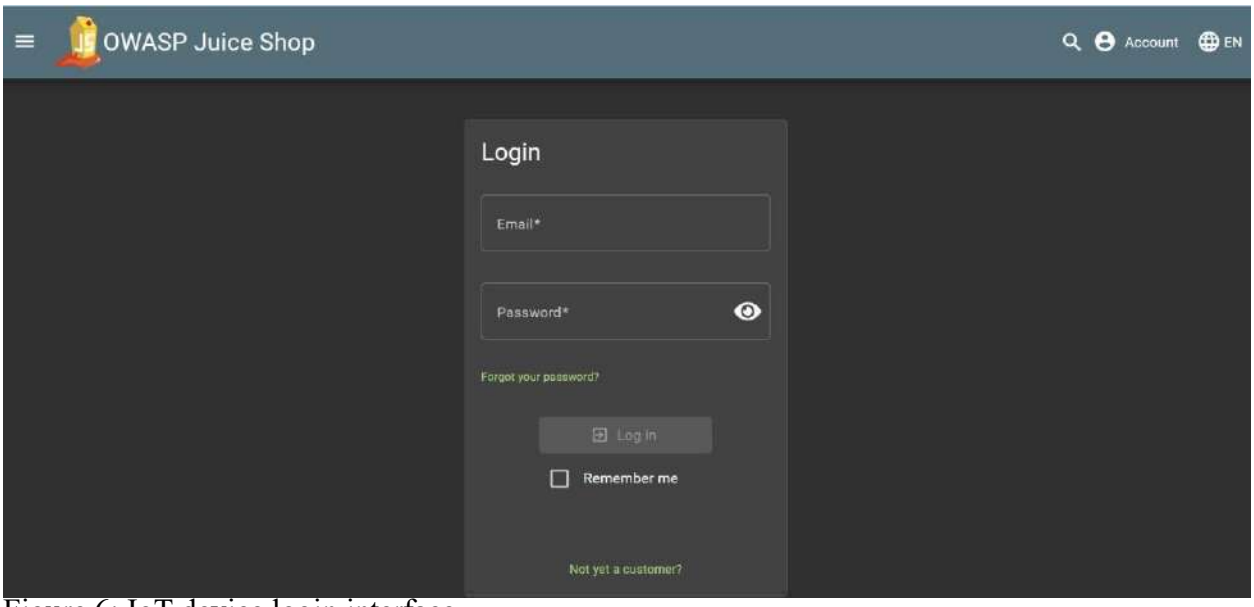


Figure 6: IoT device login interface

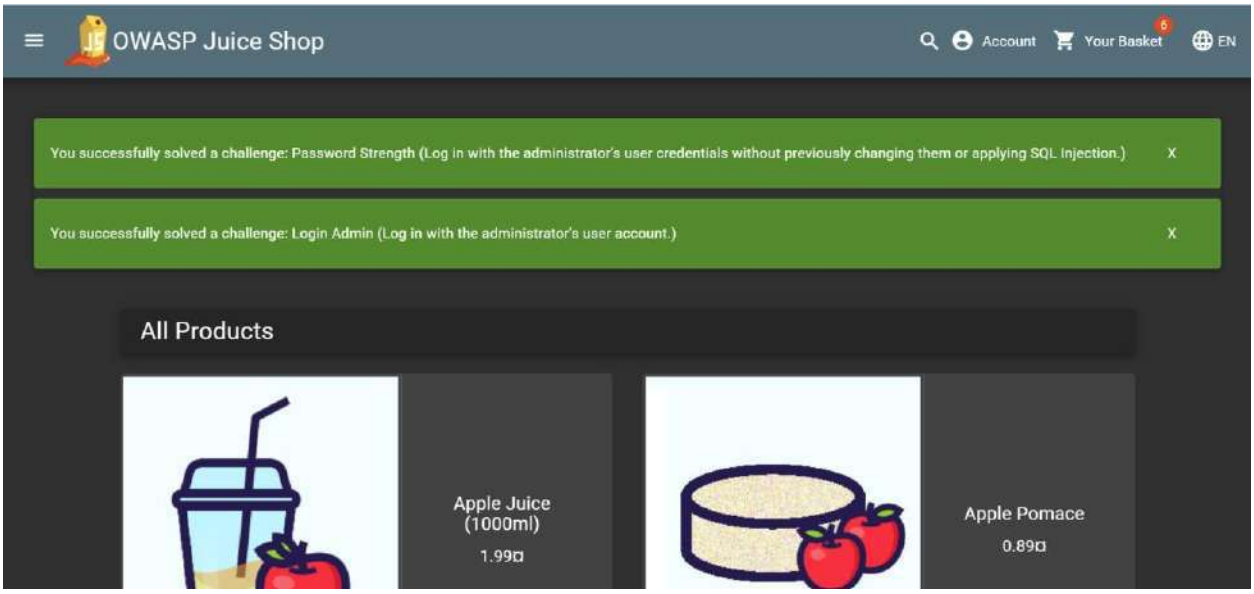


Figure 7: Successful login using default/weak credentials

Successful login using administrator credentials demonstrates the risk of default or weak passwords in IoT devices. If default credentials are not changed after deployment, attackers can gain unauthorized administrative access.

Step 7: Developer Tools → Network Tab

- Network tab showing HTTP requests
- Plain text traffic visible

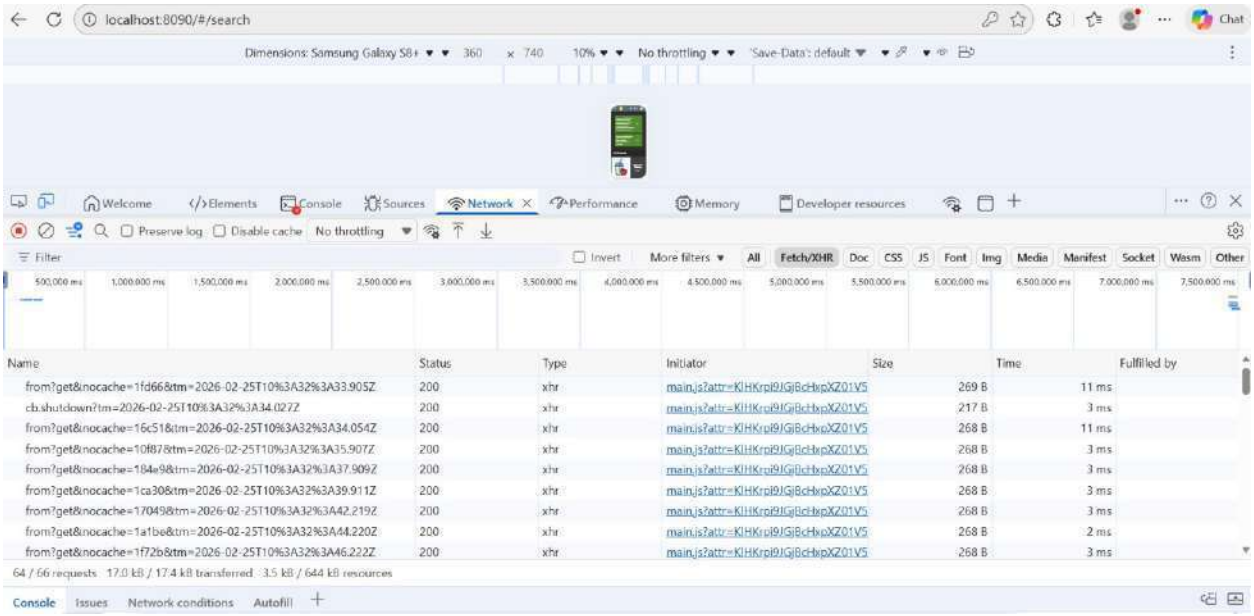


Figure 8: Unencrypted HTTP communication analysis

Observations

The scan revealed:

- Port 8090 open (HTTP service running).
- Service fingerprint identified as Juice Shop.
- Windows system ports (135, 139, 445) visible.
- Service version information exposed.
- MAC address detected.
- Web communication running over HTTP (not HTTPS).

Using Browser Developer Tools:

- Network requests visible in plain text.
- No SSL/TLS encryption detected.

Vulnerabilities Identified

1. **Open Port Exposure**
   - Port 8090 accessible over network.
   - Increases attack surface.
2. **Service Enumeration**
   - Service version and fingerprint visible.
   - Enables targeted exploit attempts.
3. **Unencrypted Communication**
   - Application runs over HTTP.
   - Data transmitted in plain text.
4. **Weak Authentication Risk**
   - Login interface exposed.
   - No strict account lockout mechanism.
5. **Information Disclosure**
   - Banner reveals application identity.
   - Server headers visible.

Conclusion

The experiment demonstrated how an attacker on the same network can identify open ports, detect running services, and test default credentials on an IoT device. The presence of exposed services and unencrypted communication highlights common IoT security risks. Proper configuration, strong authentication, and secure communication protocols are essential to protect IoT devices from exploitation.

Experiment 17: Creating and Analyzing Disk Images Using dc3dd and Autopsy (Alternative to FTK Imager)

1. Aim

To create a forensic disk image and analyze digital evidence using **dc3dd** and **Autopsy** in **Kali Linux**.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of **digital forensics and disk imaging**
- Create a **bit-by-bit forensic copy of a storage device**
- Maintain **data integrity using hashing**
- Analyze disk images to identify **files, deleted files, and system artifacts**

3. Introduction /

Theory Digital

Forensics

Digital forensics is the process of **identifying, collecting, preserving, analyzing, and presenting digital evidence** from computers or storage devices.
Digital evidence may include:

- Documents
- Emails
- Browser history
- Logs
- Deleted files

To preserve evidence, investigators create a **forensic disk image**.

Disk Imaging

Disk imaging is the process of creating an **exact replica of a storage device**.

Characteristics of forensic disk images:

- Bit-by-bit copy of storage
- Preserves deleted files
- Maintains data integrity
- Allows investigation without altering original evidence

Hashing in Forensics

Hashing is used to verify the **integrity of digital evidence**.

Common hash algorithms:

Algorithm	Purpose
MD5	evidence integrity
SHA1	verification
SHA256	stronger security

If the hash value of the original disk and the disk image match, the **evidence is considered intact**.

4. Tools Used

Tool	Purpose
Kali Linux	forensic analysis environment
dc3dd	disk image acquisition
Autopsy	forensic investigation and analysis

5. System Requirements

Component	Specification
Processor	Intel i3 or higher
RAM	Minimum 4 GB
Disk	20 GB free space
Software	Kali Linux VM

6. Algorithm

1. Start the Kali Linux environment.
2. Generate activity on the system to create forensic artifacts.
3. Identify the disk partition using system commands.
4. Create a forensic disk image using dc3dd.
5. Generate hash values for integrity verification.
6. Start the Autopsy forensic analysis tool.
7. Import the disk image into Autopsy.
8. Analyze files, logs, and deleted artifacts.
9. Extract relevant digital evidence.

7. Procedure

Step 1: Open Terminal in Kali Linux

Press:

Ctrl + Alt + T

This opens the terminal.

Step 2: Create Sample Evidence

Generate a file that will later be discovered during forensic analysis.

```
echo "Cybersecurity Lab Evidence" > evidence.txt
```

Verify the file creation.

```
Ls
```

Step 3: Identify Disk Partition

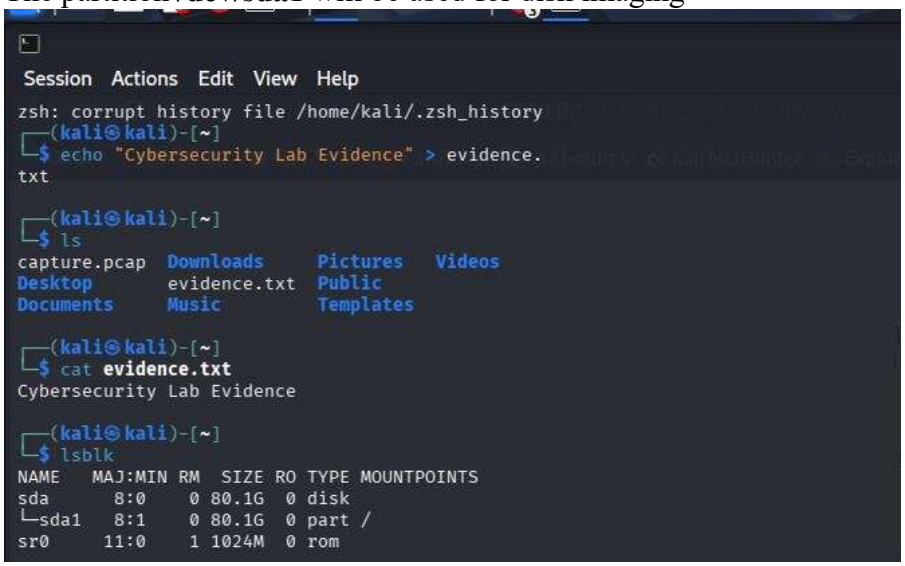
List storage devices.

```
lsblk
```

Example output:

```
sda
 ├─sda1
 ├─sda2
```

The partition **/dev/sda1** will be used for disk imaging



Install dc3dd

```
┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ sudo apt install dc3dd
Installing:
  dc3dd

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not U
pgrading: 1757
  Download size: 117 kB
  Space needed: 489 kB / 60.2 GB available

Get:1 http://kali.download/kali kali-rolling/main
 amd64 dc3dd amd64 7.3.1-4 [117 kB]
Fetched 117 kB in 5s (21.9 kB/s)
Selecting previously unselected package dc3dd.
(Reading database ... 454289 files and directorie
s currently installed.)
Preparing to unpack .../dc3dd_7.3.1-4_amd64.deb .
..
Unpacking dc3dd (7.3.1-4) ...
Setting up dc3dd (7.3.1-4) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

┌──(kali㉿kali)-[~]
└─$ dc3dd --help
usage:

        dc3dd [OPTION 1] [OPTION 2] ... [OPTION N
]

                *or*

        dc3dd [HELP OPTION]

        where each OPTION is selected from the ba
sic or advanced
        options listed below, or HELP OPTION is s
elected from the
        help options listed below.
```

Step 4: Create Disk Image

Step 1: Create a Small Disk File

dd if=/dev/zero of=practice_disk.dd bs=1M count=100

```
┌──(kali㉿kali)-[~]
└─$ dd if=/dev/zero of=practice_disk.dd bs=1M cou
nt=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.28316
5 s, 370 MB/s
```

Explanation:

Parameter	Description
if	input disk
of	output image file
hash	generate MD5 hash
log	acquisition log file

Creates a **100 MB disk**.

Step 2: Mount the Disk

mkfs.ext4 practice_disk.dd

```
┌──(kali㉿kali)-[~]
└─$ mkfs.ext4 practice_disk.dd
mke2fs 1.47.2 (1-Jan-2025)
Discarding device blocks: done

Creating filesystem with 102400 1k blocks and 255
84 inodes
Filesystem UUID: 514f4c47-066d-4a31-8af3-f13b33b7
5b90
Superblock backups stored on blocks:
        8193, 24577, 40961, 57345, 73729

Allocating group tables: done

Writing inode tables: done

Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting inf
ormation: done
```

Step 5: Verify Image Creation

Check the created image.

ls -lh /home/kali/disk_image.dd

```
┌──(kali㉿kali)-[~]
└─$ ls -lh /home/kali/disk_image.dd
-rw-r--r-- 1 root root 9.8G Mar 15 11:59 /home/ka
li/disk_image.dd
```

Check log file.

cat /home/kali/acquisition.log

The log will contain:

- image size
- hash value
- acquisition details

```
┌──(kali㉿kali)-[~]
└─$ cat /home/kali/acquisition.log

dc3dd 7.2.646 started at 2026-03-15 11:52:30 -040
0
compiled options:
command line dc3dd if=/dev/sda1 of=/home/kali/dis
k_image.dd hash=md5 log=/home/kali/acquisition.lo
g
device size: 167966702 sectors (probed),   85,998
,951,424 bytes
sector size: 512 bytes (probed)
 10503356416 bytes ( 9.8 G ) copied ( 12% ), 432.
546 s, 23 M/s

input results for device `/dev/sda1':
   20514368 sectors in
   0 bad sectors replaced by zeros
   ce827a581c4dfce5b976603df85bb5e0 (md5)

output results for file `/home/kali/disk_image.dd
':
   20514368 sectors out

dc3dd aborted at 2026-03-15 11:59:42 -0400
```

Step 6: Start Autopsy

Launch the forensic analysis tool.

Autopsy

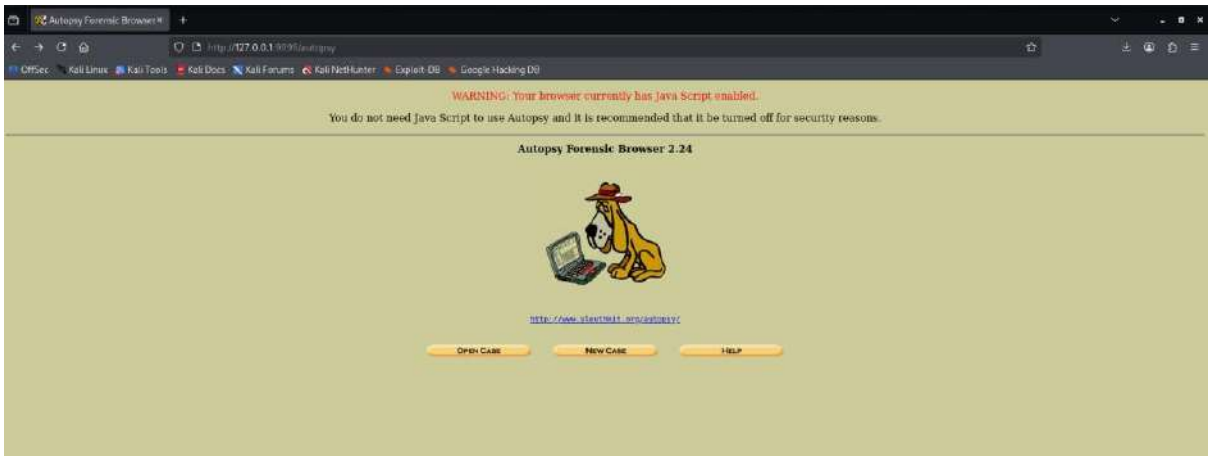
```
┌──(kali㉿kali)-[~]
└─$ autopsy

=================================================
=================================================

                     Autopsy Forensic Browser
                 http://www.sleuthkit.org/autops
y/
                            ver 2.24

=================================================
=================================
Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:01:18 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste
 this URL in it:

    http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exi
```

```
┌──(kali㉿kali)-[~]
└─$ sudo autopsy
[sudo] password for kali:

Autopsy Forensic Browser
http://www.sleuthkit.org/autopsy/
ver 2.24

Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:16:57 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste this URL in it:

    http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exit
```

The terminal will display a URL:

http://localhost:9999/autopsy Open this link

in the browser.



Step 7: Create a New Case

In the Autopsy interface:

1. Click **Create New Case**
2. Enter case details:

Step 8: Add Disk Image

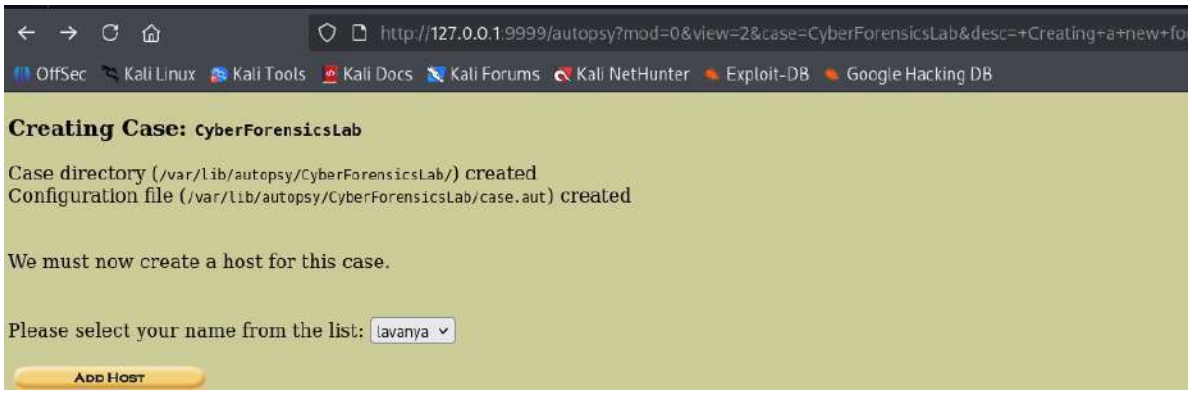
Select:

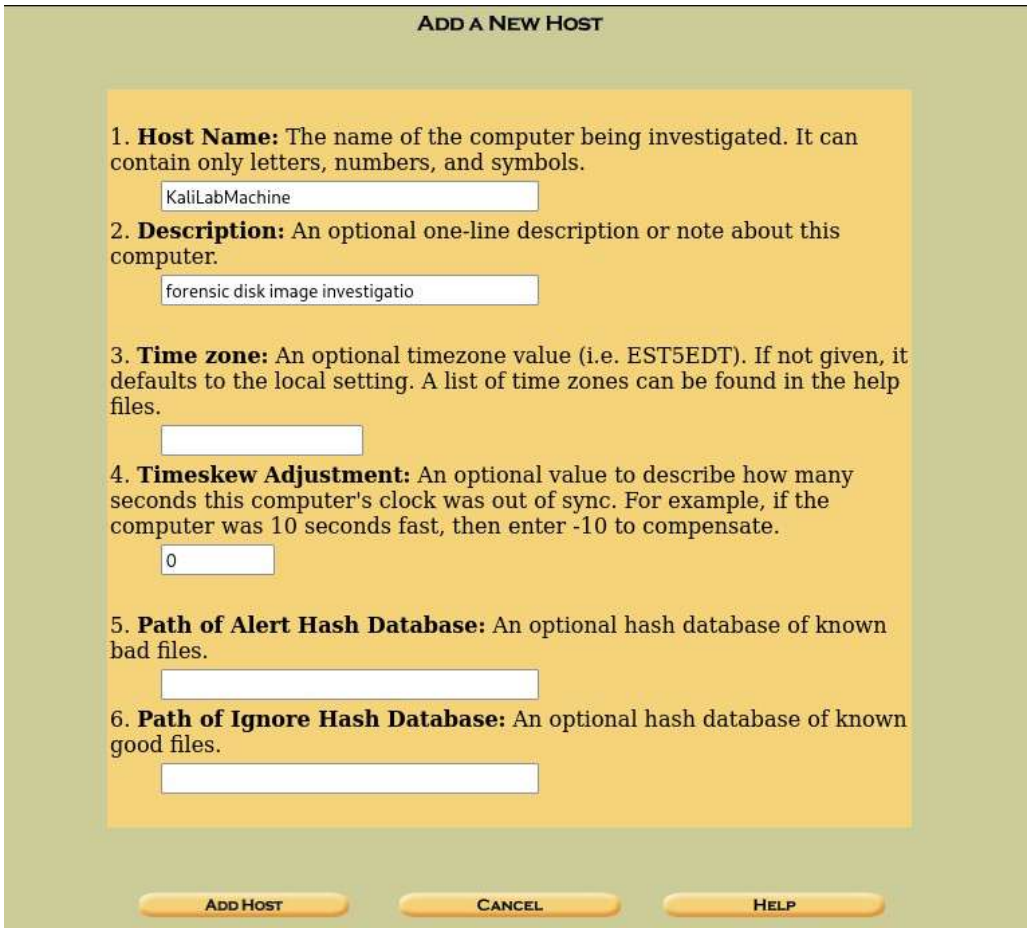
Add Host → Add Image

Browse and select:

/home/kali/disk_image.dd

Autopsy will begin analyzing the disk image.





Recommended Values

Host Name

KaliLabMachine

Description

Kali Linux virtual machine used for forensic disk image investigation

Time Zone

Leave Blank

Time Skew Adjustment

Leave Blank

Alert Hash Database

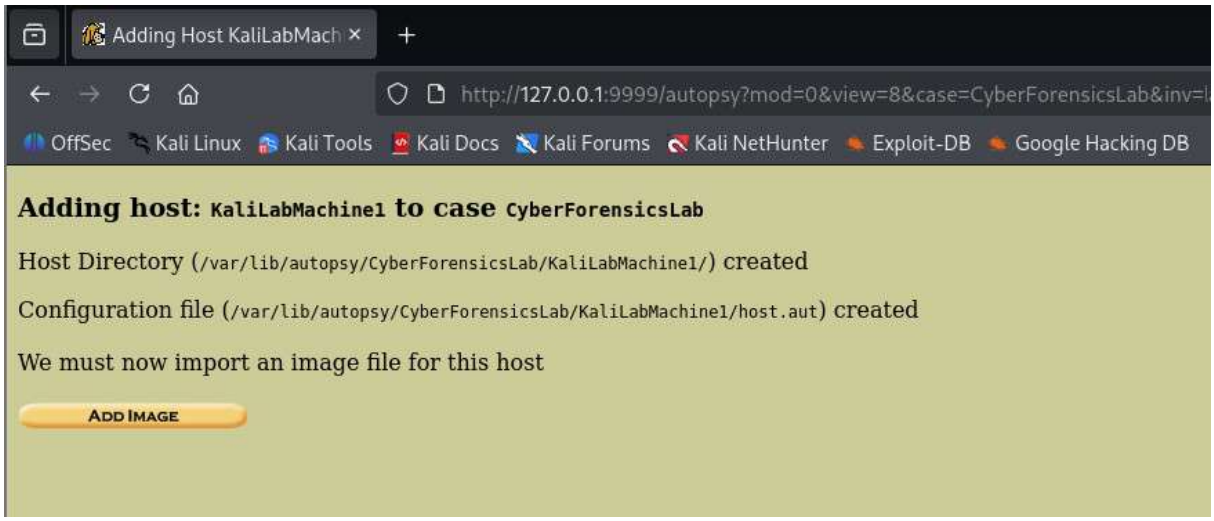
Leave Blank

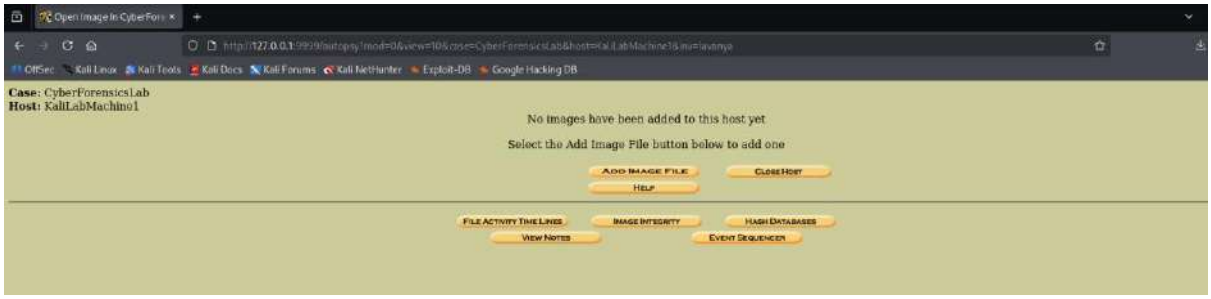
Ignore Hash Database

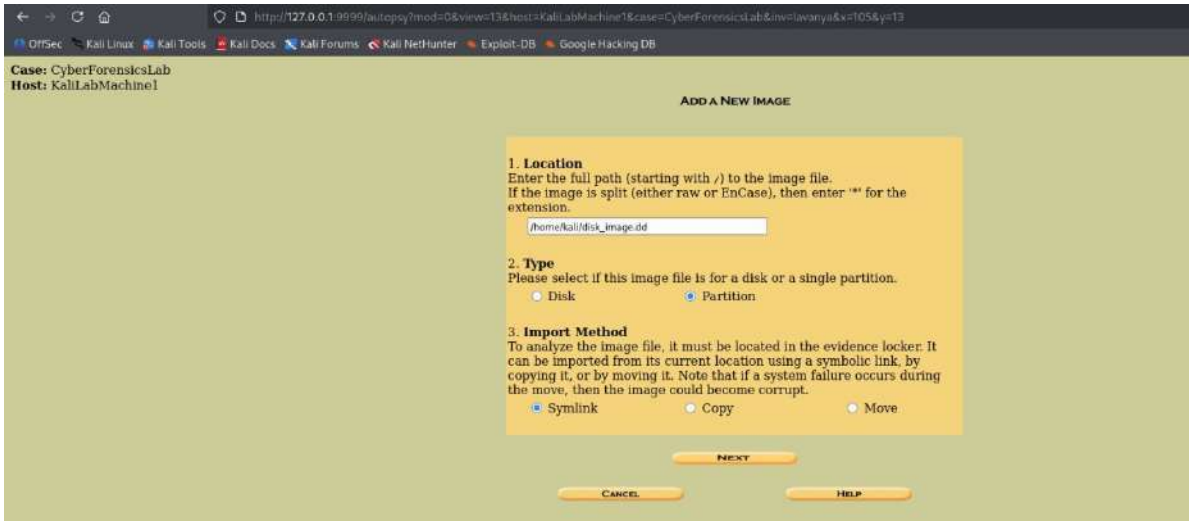
Leave Blank

Then click:

Next







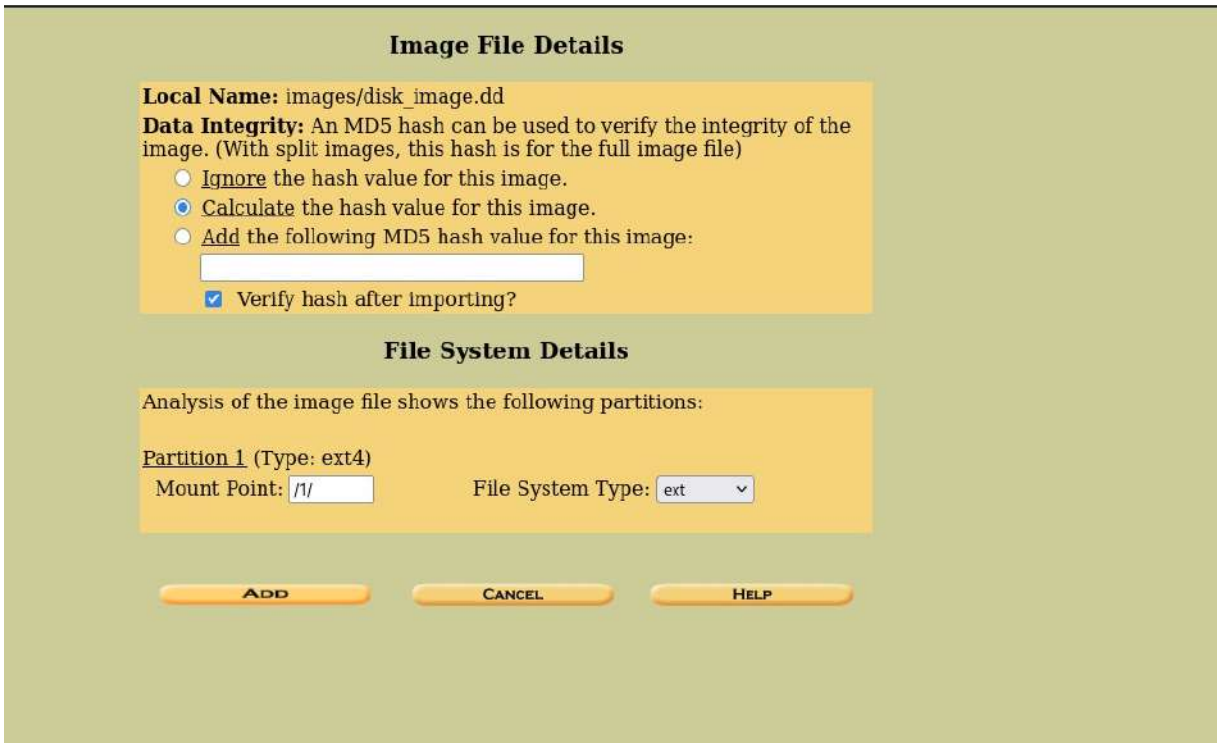
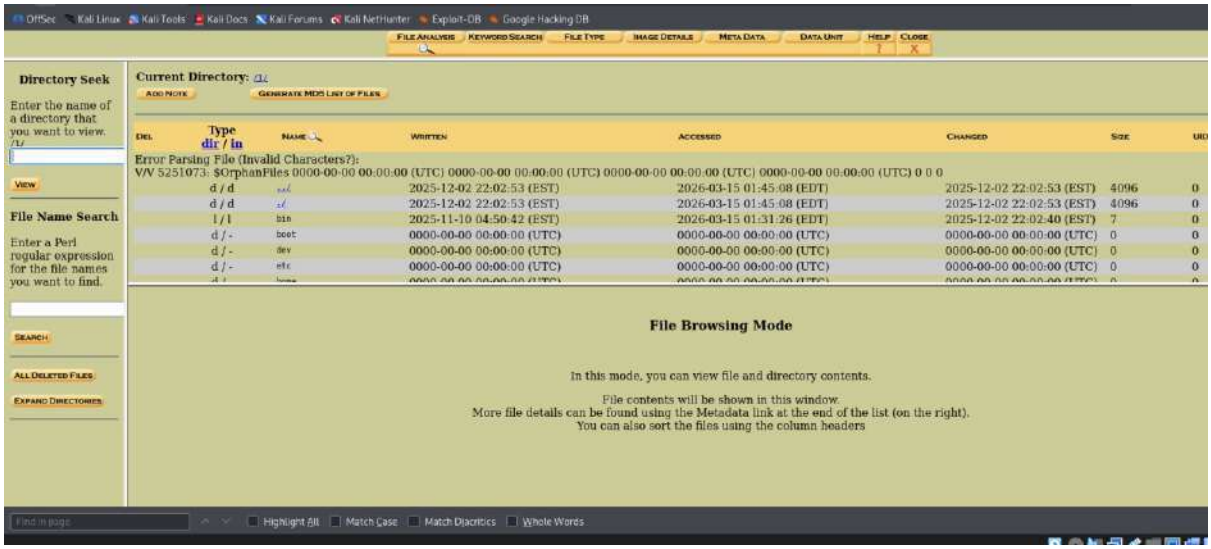


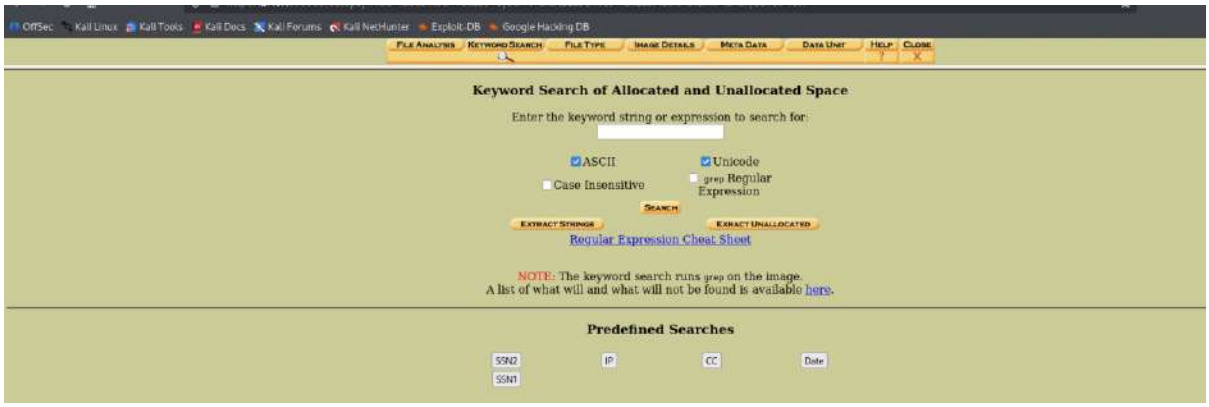
Image File Details
Local Name: images/disk_image.dd
Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)
Ignore the hash value for this image.
Calculate the hash value for this image.
Add the following MD5 hash value for this image:
Verify hash after importing?
File System Details
Analysis of the image file shows the following partitions:
Partition 1 (Type: ext4)
Mount Point: /1/
File System Type: ext
ADD
CANCEL
HELP

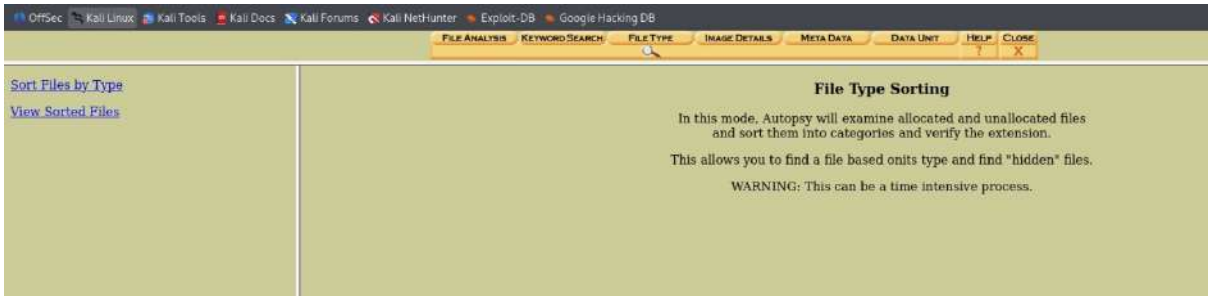
Calculating MD5 (this could take a while)
Current MD5: CE827A581C4DFCE5897660JOFB5B85E0
Testing partitions
Linking image(s) into evidence locker
Image file added with ID img1
Volume image (0 to 0 - ext - /1/) added with ID vol1
OK
ADD IMAGE

Case: CyberForensicsLab
Host: KaliLabMachine1
Select a volume to analyze or add a new image file.
CASE GALLERY
HOST GALLERY
HOST MANAGER
mount
name
fs type
/1/
disk_image.dd-0-0
ext
details
ANALYZE
ADD IMAGE FILE
CLOSE HOST
HELP
FILE ACTIVITY TIME LINES
IMAGE INTEGRITY
HASH DATABASES
VIEW NOTES
EVENT SEQUENCER

http://127.0.0.1:9999/autopsy?mod=0&view=17&host=KaliLabMachine1&case=CyberForensicsLab&inv=lavanya&vol=vol1&x=78&y=16
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
FILE ANALYSIS
KEYWORD SEARCH
FILE TYPE
IMAGE DETAILS
META DATA
DATA UNIT
HELP
CLOSE
To start analyzing this volume, choose an analysis mode from the tabs above.







Step 9: Analyze Evidence

Experiment 18 : LOG FILE ANALYSIS FOR INCIDENT DETECTION LAB

Aim

To analyze system log files and detect suspicious activities such as failed login attempts and identify suspicious IP addresses.

Objective

- Understand system log files
- Detect suspicious activities and incidents
- Analyze logs using Linux tools
- Identify attack patterns (failed logins, brute force, etc.)

Requirements

- Kali Linux / Ubuntu VM
- Basic Linux command knowledge
- Tools:
- Kali Linux
- journalctl
- ssh
- grep, awk, sort
- grep
- awk
- less
- cat
- journalctl

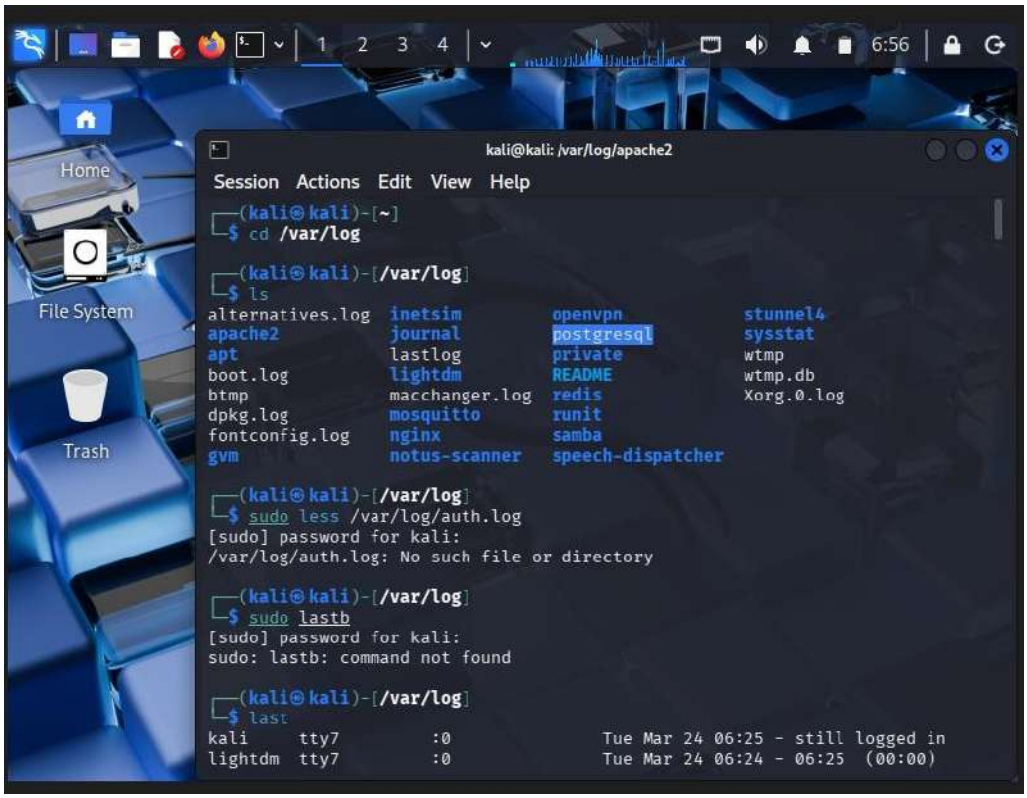
Understanding Log

Files Navigate to Log

Directory cd /var/log

ls

The /var/logdirectory contains system and application logs such as Apache, Nginx, and login records used for security analysis.



What Your Output

Shows From your ls:

- ✔ **dpkg.log → package activity**
- ✔ **apache2 → web server logs**
- ✔ **nginx → web logs**
- ✔ **journal → system logs (IMPORTANT)**
- ✔ **wtmp, btmp → login records**

Successful Logins (wtmp)

Command:

last

Capture:

- Login history

This displays successful login sessions. It helps identify legitimate and suspicious user access.

System Logs using journalctl

Command:

journalctl | less

Capture:

- Logs output

Failed SSH Attempts

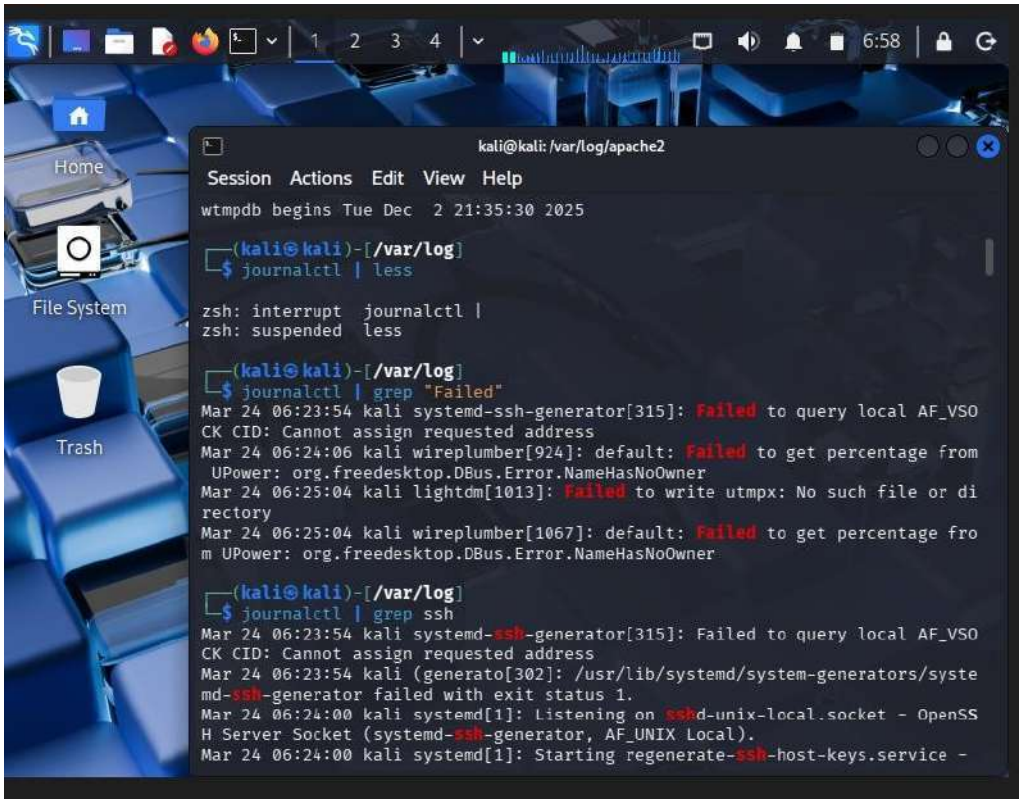
Command:

journalctl | grep "Failed"

SSH Login Activity

Command:

journalctl | grep ssh

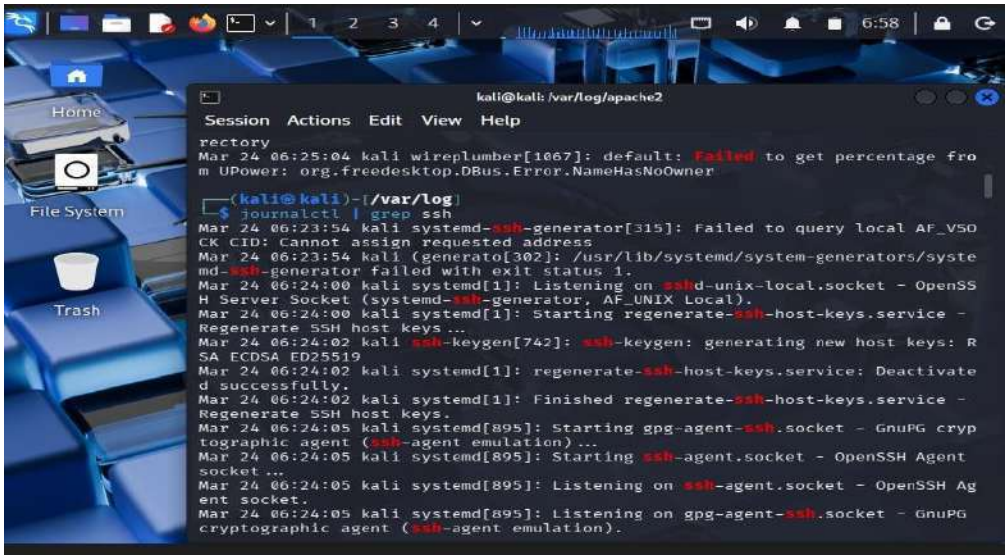


Explanation:

The journalctl command provides centralized system logs including authentication, services, and system events.

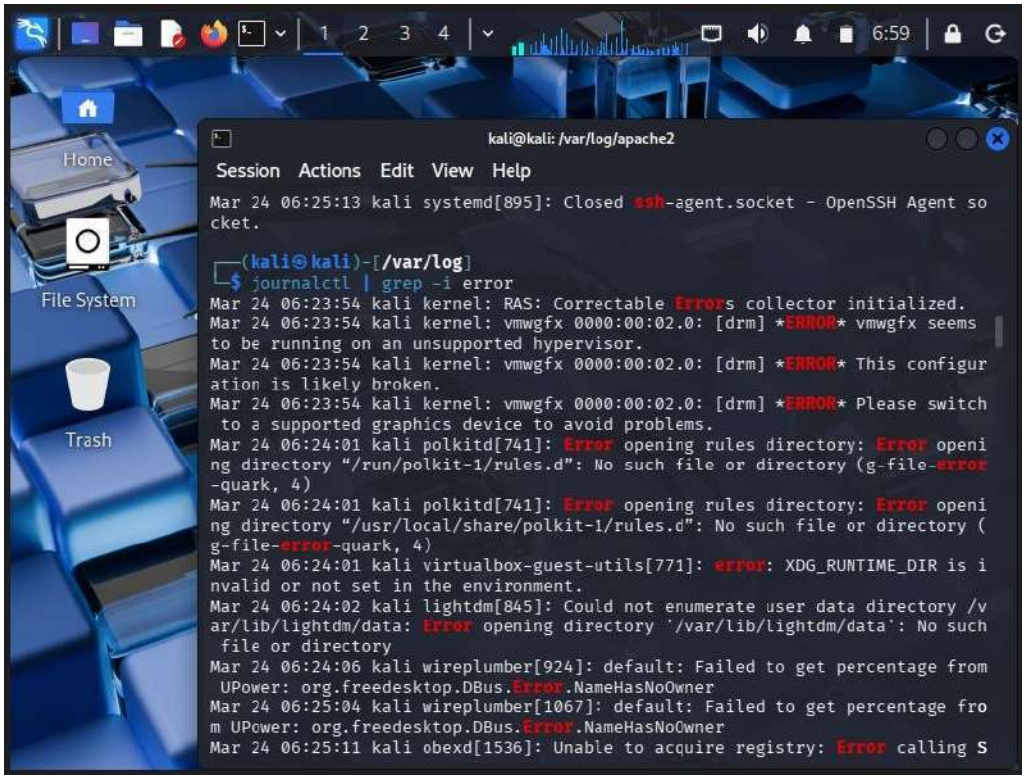
This screenshot shows failed authentication attempts, which may indicate unauthorized access attempts.

This screenshot also displays SSH-related logs, including login attempts and connections.



Error Detection

Command: journalctl | grep -i error



The above screenshot highlights system errors, which may indicate abnormal behavior or system issues.

Apache Log Analysis

Command:

cd /var/log/apache2 ls

Then:

sudo less access.log
Apache logs record web requests. These logs help identify suspicious web activity such as repeated requests or attacks.

Suspicious Requests Command:

grep "404" /var/log/apache2/access.log
This shows failed web requests (404 errors), which may indicate scanning or probing attacks.

Package Activity

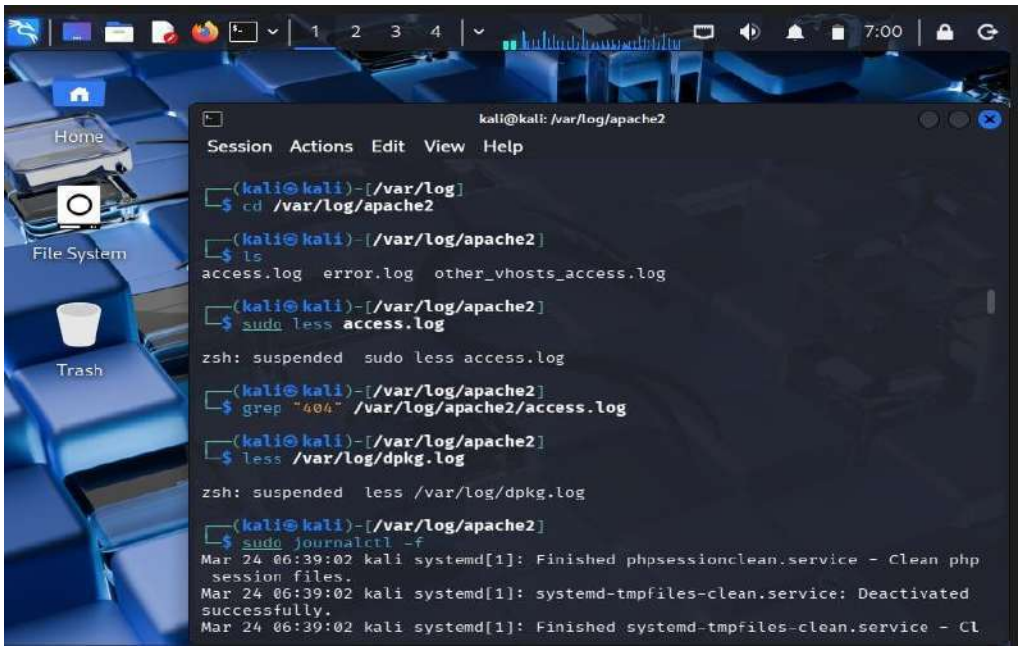
Command:

less /var/log/dpkg.log
This shows software installation and update logs, useful for detecting unauthorized changes.

Real-Time Monitoring Command:

sudo journalctl -f

This shows real-time system log monitoring, allowing detection of incidents as they occur.



QUICK CHECKLIST

Check	Command	Suspicious If
Failed logins	lastb	Many attempts
SSH activity	`journalctl	grep ssh`

Login history	last	Odd timing
Errors	`journalctl	grep error`
Web logs	grep 404	Many failed requests

STEP-BY-STEP ANALYSIS OF YOUR LOG

1. System Services (NORMAL)

Finished phpsessionclean.service Finished systemd-tmpfiles-clean.service

✔ These are routine background tasks
✔ No issue
Regular system maintenance services were observed, indicating normal system operation.

2. CRON Jobs (NORMAL)

CRON: session opened for user root (root) CMD (debian-sa1)

✔ Automated scheduled task
✔ Runs periodically

Scheduled cron jobs were observed, which are normal automated system processes.

3. Your Own SUDO Activity (NORMAL)

kali : COMMAND=/usr/bin/less access.log kali : COMMAND=/usr/bin/journalctl -f

✔ This is YOU running commands
✔ Not suspicious

Sudo activity corresponds to legitimate administrative actions performed by the user.

4. XFCE / GUI Services (NORMAL)

xfconfd.service started

✔ Desktop environment service
✔ Normal

5. IMPORTANT PART

● Repeated Kernel Messages

dndHGCM: Received message HOST_DND_FN_GH_REQ_PENDING dnd: No guest source window

These lines are:

Repeated MANY times
Coming from VirtualBox (VBoxDRM / DnD)

Q Is This Suspicious?

Technically:

- NOT an attack
- But **abnormal repetitive behavior**

! **Why it matters:**

- Repeated logs = unusual system activity
- Could indicate:
  - Misconfiguration
  - Resource issue
  - VirtualBox interaction problem

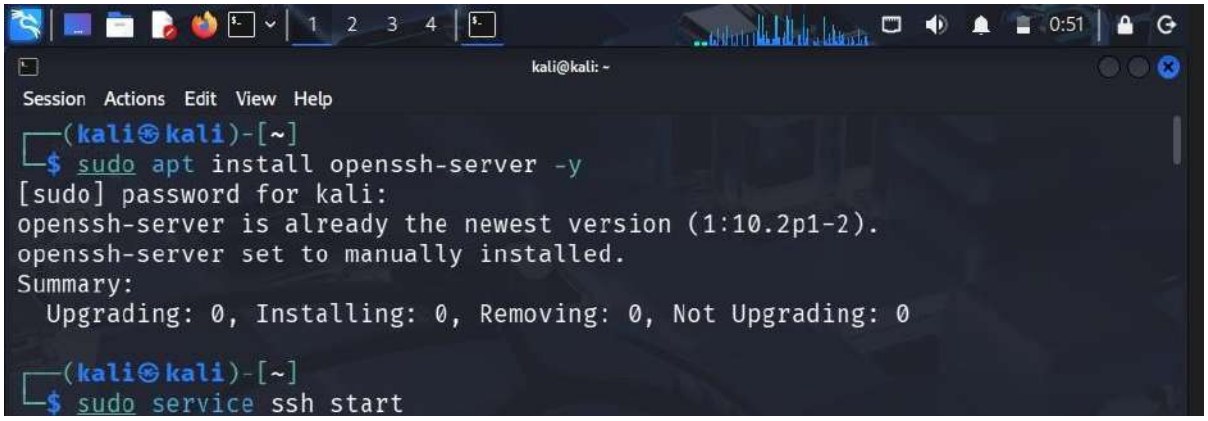
METHOD 1: SIMULATE FAILED SSH LOGIN

Step 1: Ensure SSH is Installed

sudo apt install openssh-server –y

Step 2: Start SSH Service

sudo service ssh start



Step 3: Find Your IP Address

ip a

Look for something like:

inet 192.168.x.x

```
kali@kali: ~
Session  Actions  Edit  View  Help
└─$ sudo service ssh start

┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86314sec preferred_lft 86314sec
    inet6 fd17:625c:f037:2:40d5:a10a:ee8e:32d5/64 scope global dynamic nopref
ixroute
       valid_lft 86316sec preferred_lft 14316sec
    inet6 fe80::face:768f:85f1:102f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$ sudo service ssh start
```

Step 4: Attempt Wrong Login (GENERATE FAILURES)

run:

ssh fakeuser@localhost

OR

ssh kali@localhost

Enter **wrong password multiple times (5–10 times)**

```
┌──(kali㉿kali)-[~]
└─$ ssh fakeuser@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:O2cBRrk1U0rgESjqmRmFGr8mpHGtotnR7iq/SEGToD
o
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: no
Host key verification failed.

┌──(kali㉿kali)-[~]
└─$ ssh kali@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:O2cBRrk1U0rgESjqmRmFGr8mpHGtotnR7iq/SEGToD
o
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
kali@localhost: Permission denied (publickey,password).
```

Now Analyze Logs

Step 5: Check Failed Logins

journalctl | grep "Failed password"

👉 You should now see:

Failed password for kali from 127.0.0.1 port ...

Step 6: Extract Suspicious IP

journalctl | grep "Failed password" | awk '{print $11}'

Output:

127.0.0.1

```
┌──(kali㉿kali)-[~]
└─$ ssh fakeuser@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:O2cBRrk1U0rgESjqmRmFGr8mpHGtotnR7iq/SEGToD
o
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: no
Host key verification failed.

┌──(kali㉿kali)-[~]
└─$ ssh kali@localhost
The authenticity of host 'localhost (::1)' can't be established.
ED25519 key fingerprint is: SHA256:O2cBRrk1U0rgESjqmRmFGr8mpHGtotnR7iq/SEGToD
o
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
Permission denied, please try again.
kali@localhost's password:
kali@localhost: Permission denied (publickey,password).
```

Step 7: Count Attempts Per IP

journalctl | grep "Failed password" | awk '{print $11}' | sort | uniq -c | sort -nr

Example:

10 127.0.0.1

```
└─$ journalctl | grep "Failed password"
Mar 26 00:42:21 kali sshd-session[3399]: Failed password for kali from ::1 po
rt 36238 ssh2
Mar 26 00:42:29 kali sshd-session[3399]: Failed password for kali from ::1 po
rt 36238 ssh2
Mar 26 00:42:37 kali sshd-session[3399]: Failed password for kali from ::1 po
rt 36238 ssh2

┌──(kali㉿kali)-[~]
└─$ journalctl | grep "Failed password" | awk '{print $11}'
::1
::1
::1

┌──(kali㉿kali)-[~]
└─$ journalctl | grep "Failed password" | awk '{print $11}' | sort | uniq -c
| sort -nr
      3 ::1
```

INTERPRETATION (VERY IMPORTANT)

Multiple failed login attempts were detected from IP address 127.0.0.1, indicating a simulated brute force attack.

METHOD 2: Use lastb (Alternative)

sudo lastb

Shows failed login records

HOW TO IDENTIFY "SUSPICIOUS IP"

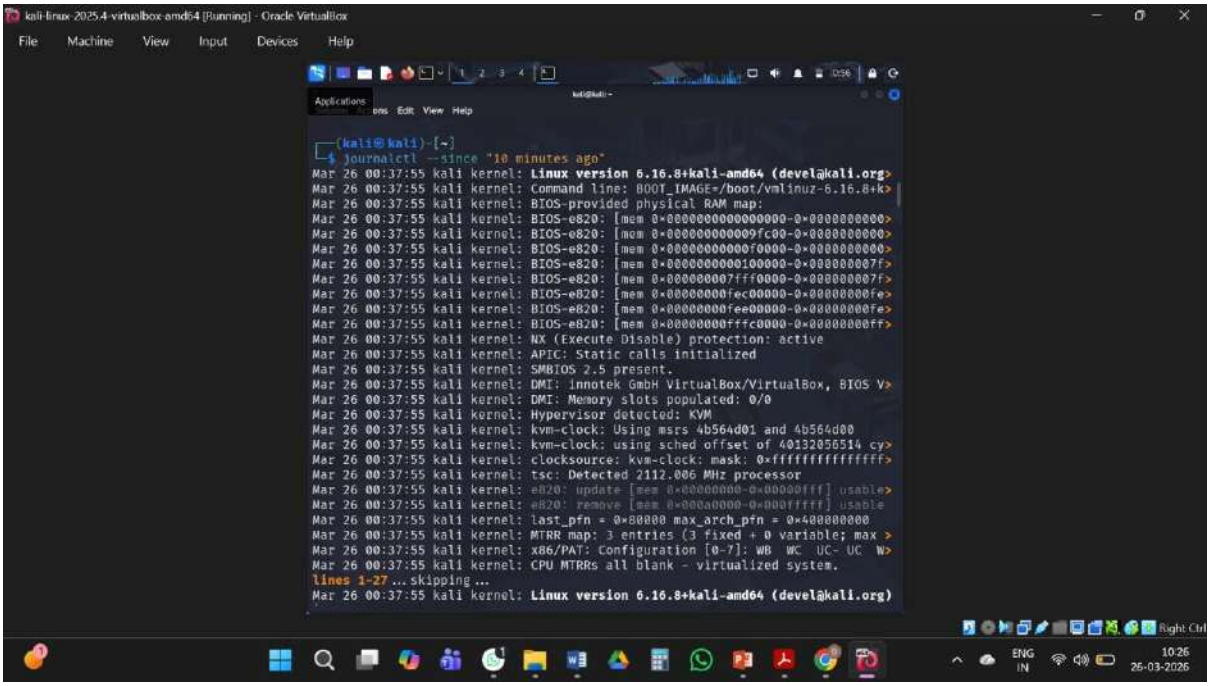
Simple rule:

Condition	Meaning
Same IP repeated many times	🚨 Suspicious
Unknown IP	🚨 Suspicious
High frequency attempts	🚨 Brute force

Run:

journalctl --since "10 minutes ago"

Shows recent attack activity



EXPERIMENT 20:

Title: Privacy Audit of Popular Apps (Desktop **WhatsApp**) and Websites (**Facebook**) & Data Breach Case Study Analysis

Aim

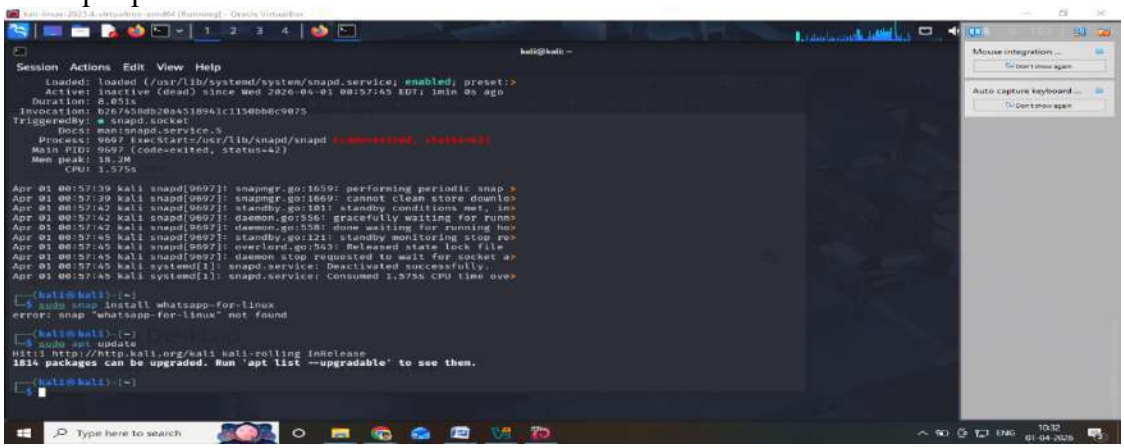
To analyze privacy, network behavior, and security risks of applications and websites using Kali Linux tools.

Tools Required

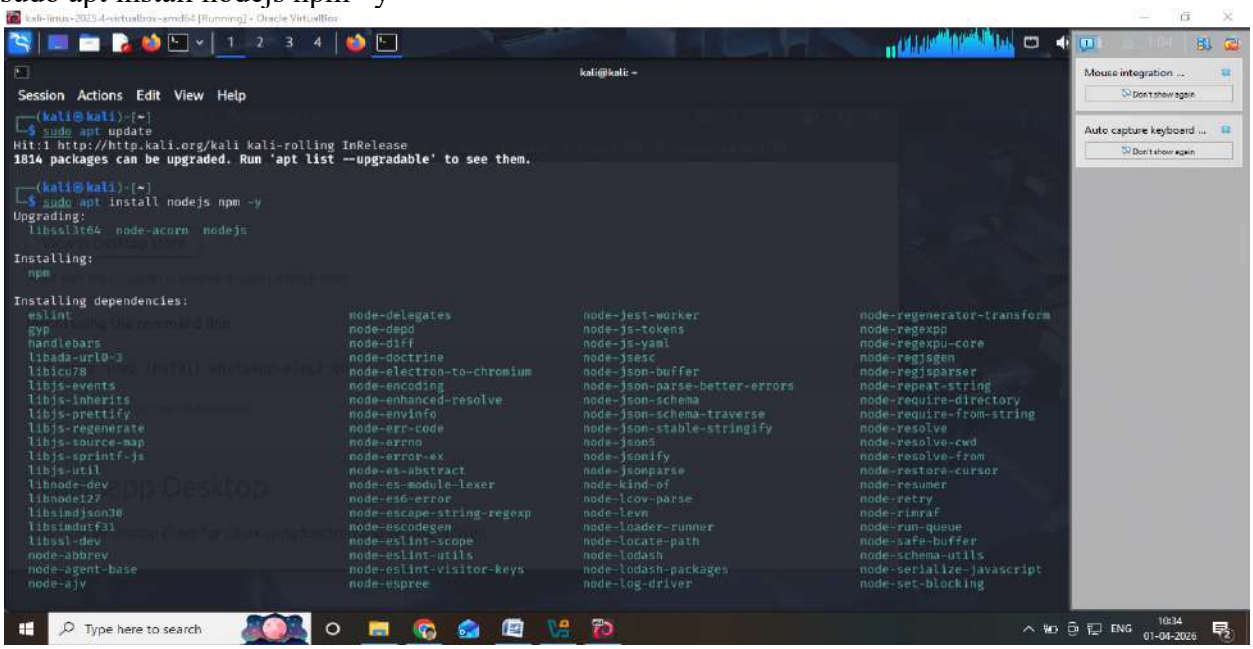
- **Wireshark**
- **Burp Suite**
- Kali Linux (VirtualBox)
- Desktop WhatsApp (Nativefier)
- Web Browser (Firefox)

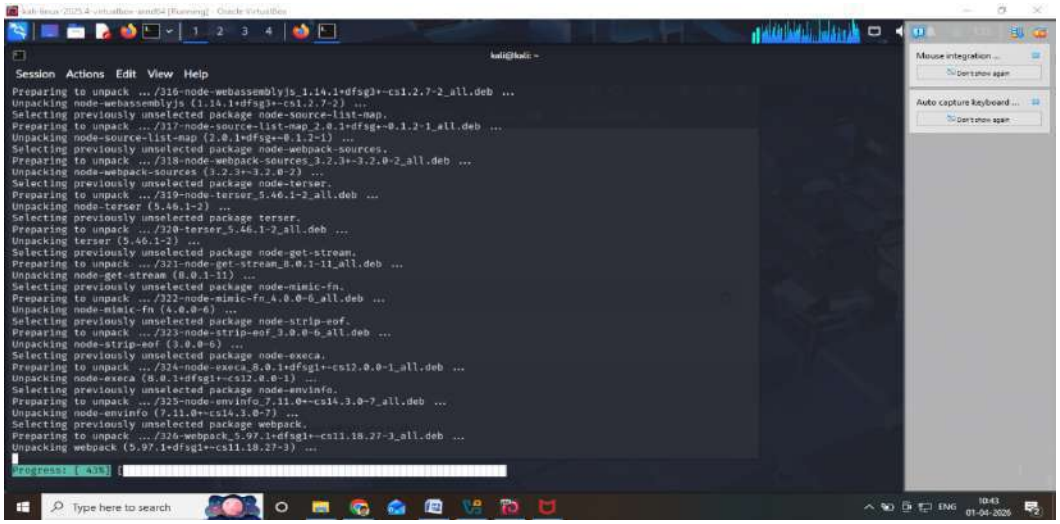
Step 1: Install Node.js & npm

sudo apt update



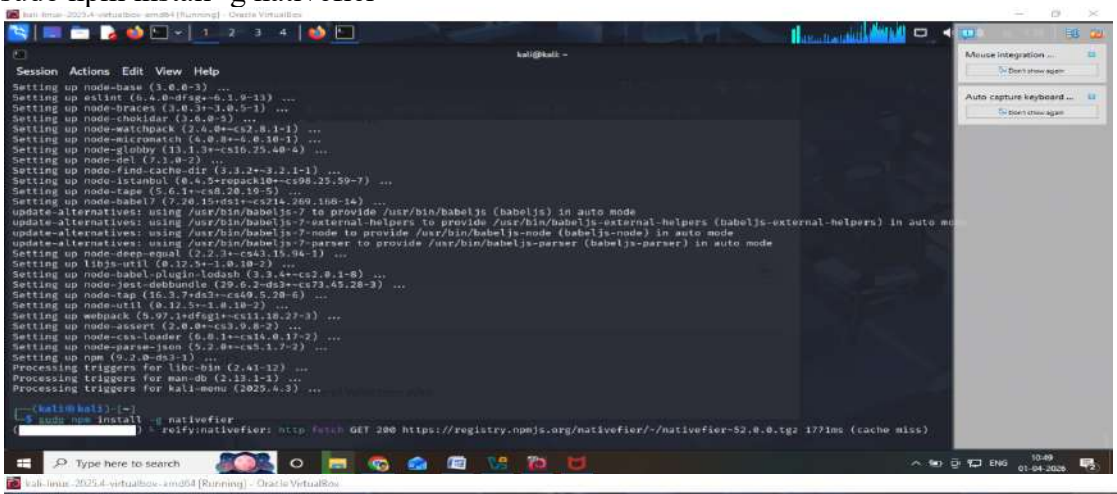
sudo apt install nodejs npm –y

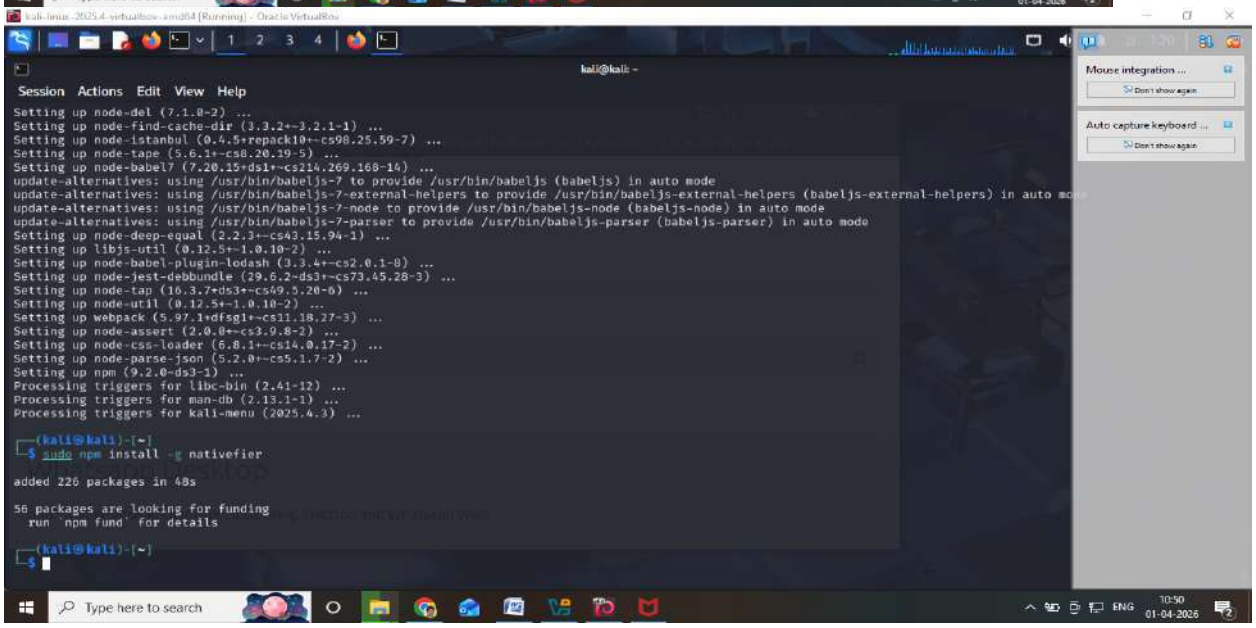




Step 2: Install Nativefier

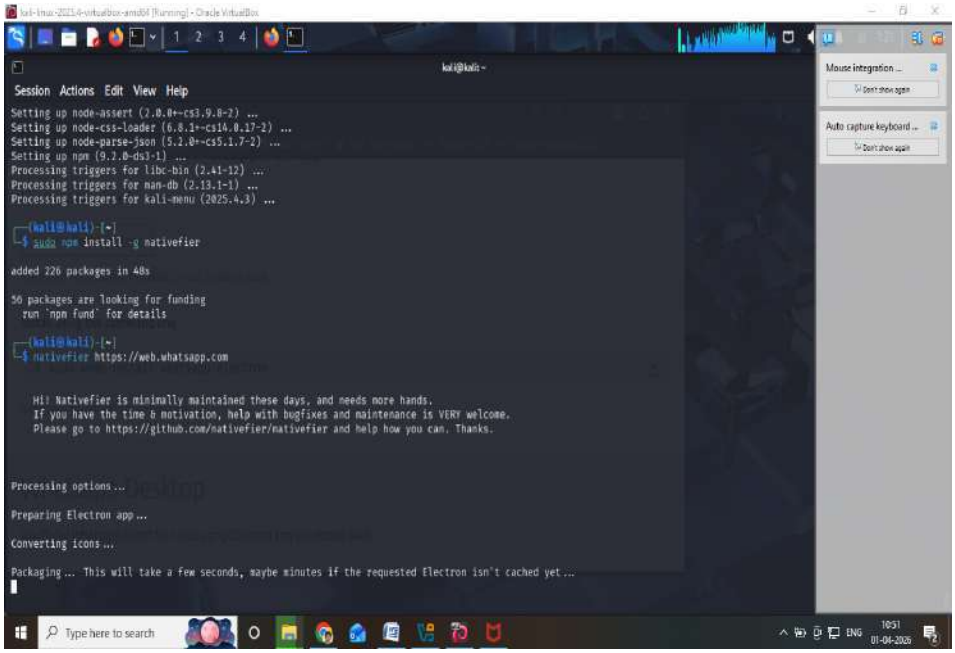
sudo npm install -g nativefier





Step 3: Create WhatsApp Desktop App

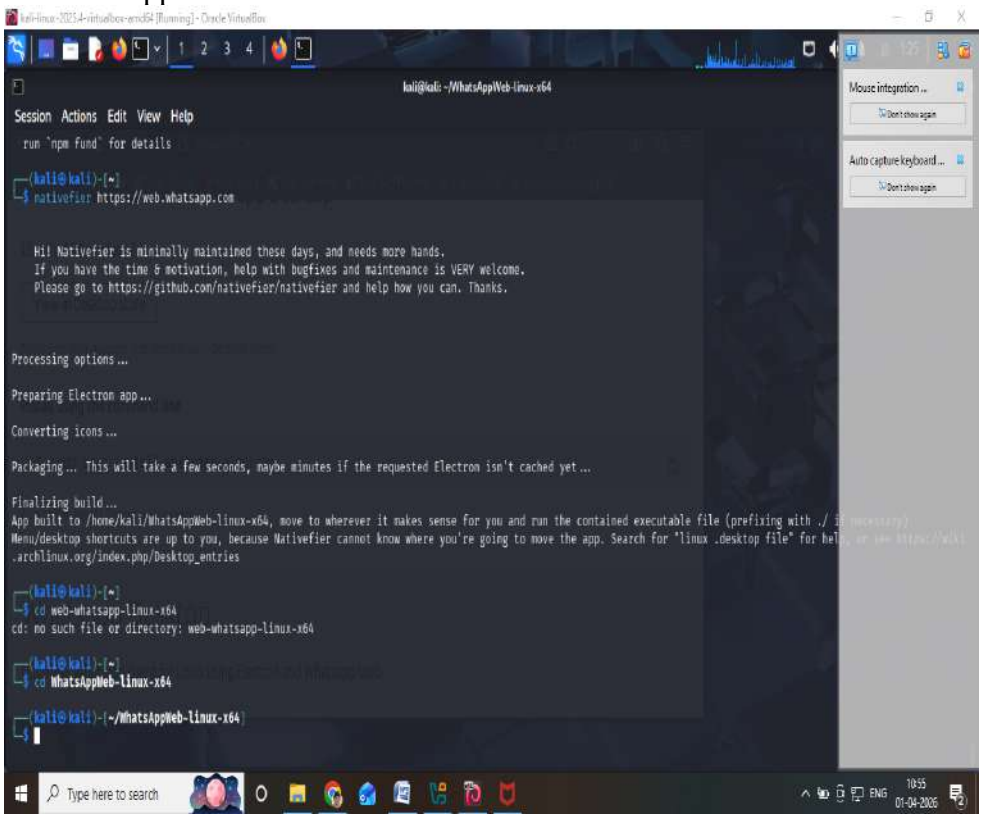
nativefier https://web.whatsapp.com



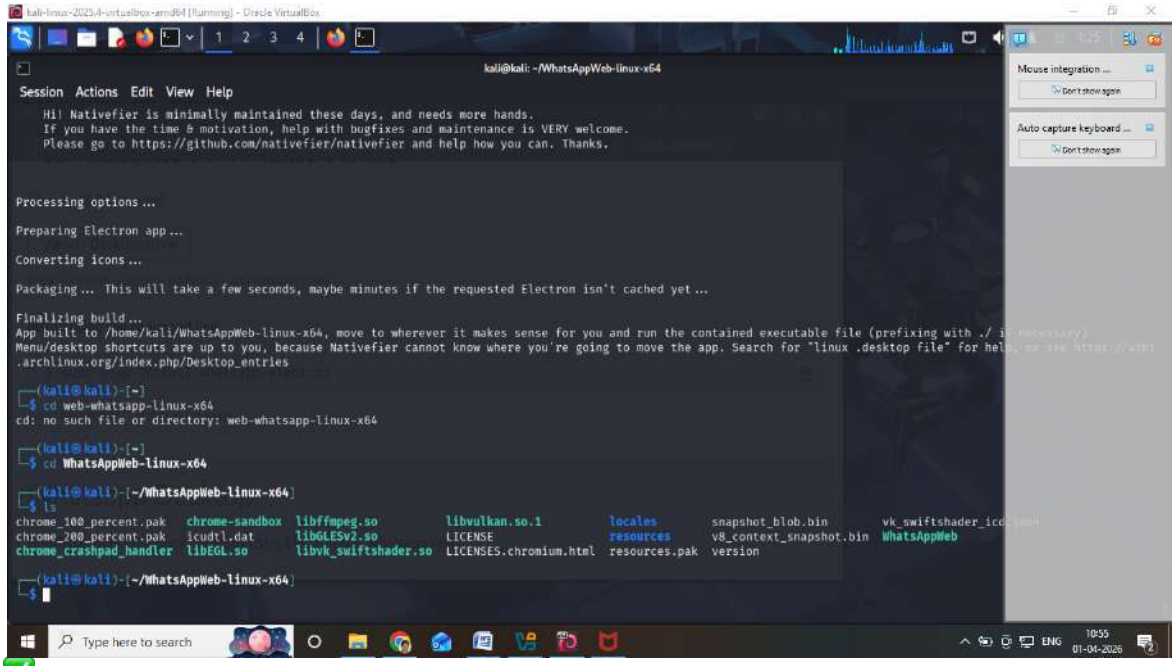
It will create a folder like:
web-whatsapp-linux-x64

Step 4: Run WhatsApp

cd WhatsAppWeb-linux-x64

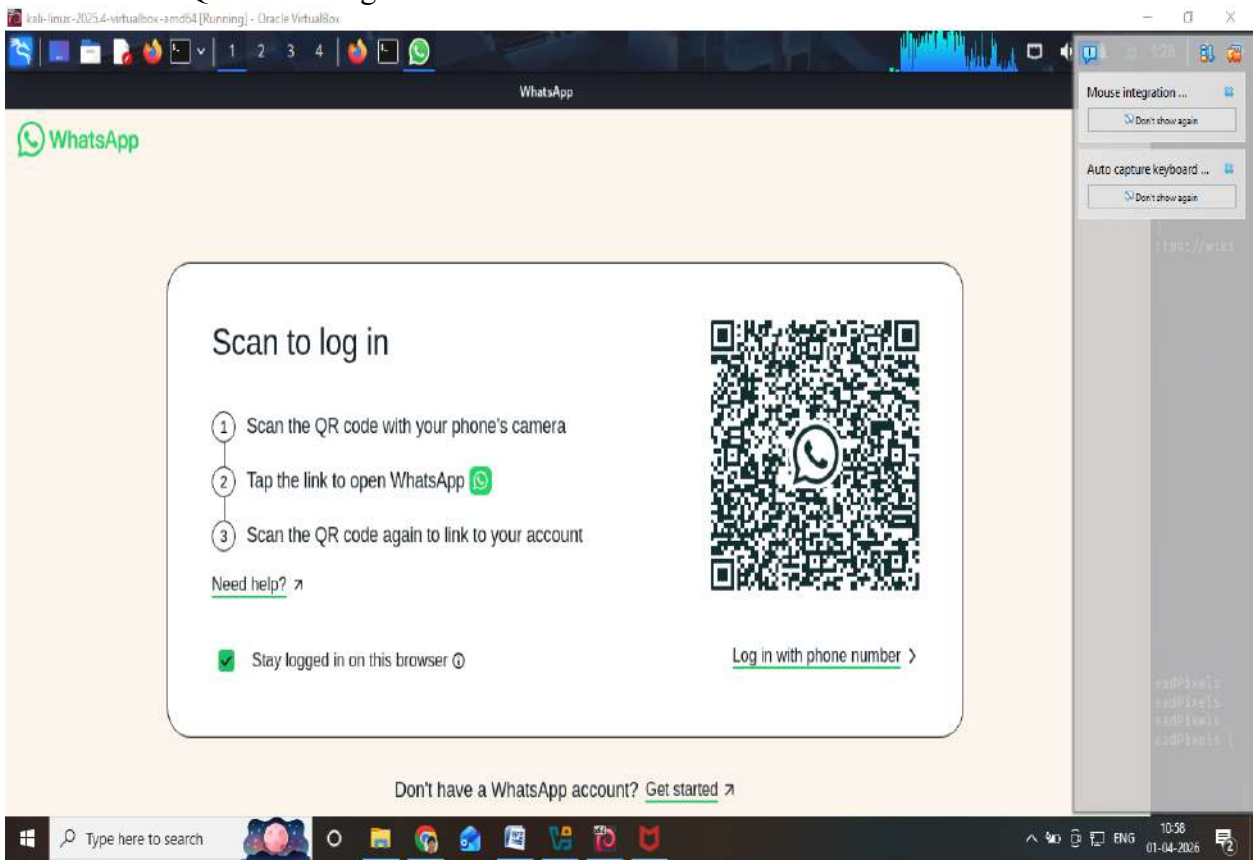


./WhatsAppWeb



✅ Step 5: Login

- Scan QR code using mobile



- Done

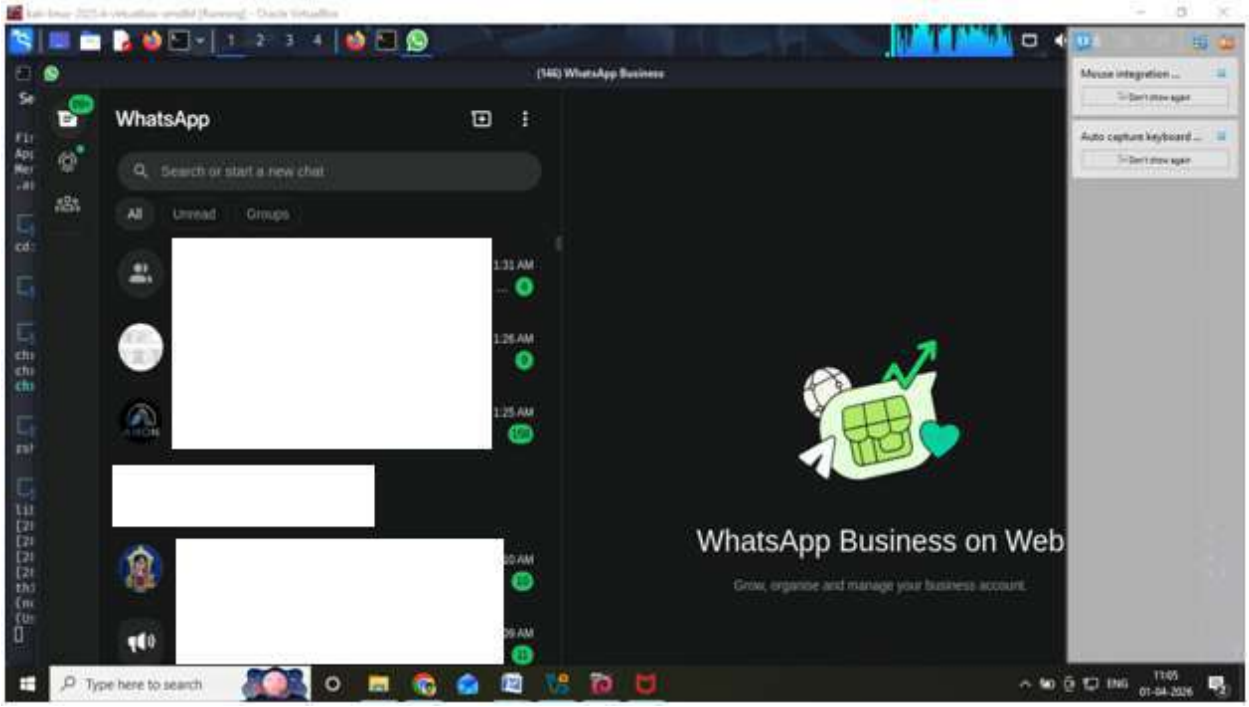
PART 1: Privacy Audit of Desktop WhatsApp

Step 1: Launch WhatsApp Desktop

cd WhatsAppWeb-linux-x64

./WhatsAppWeb

- QR Code appears
- Login using mobile → Linked Devices

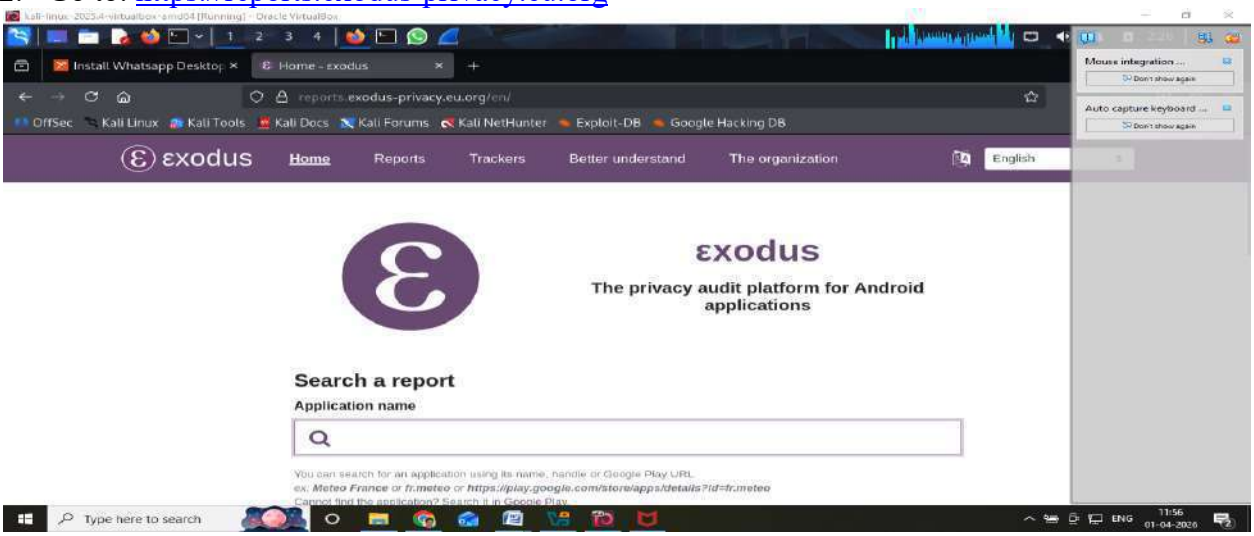


Step 1: Install WhatsApp APK (Optional in Emulator)

- Download APK from trusted source or use your mobile

Step 2: Analyze Trackers using Exodus Privacy

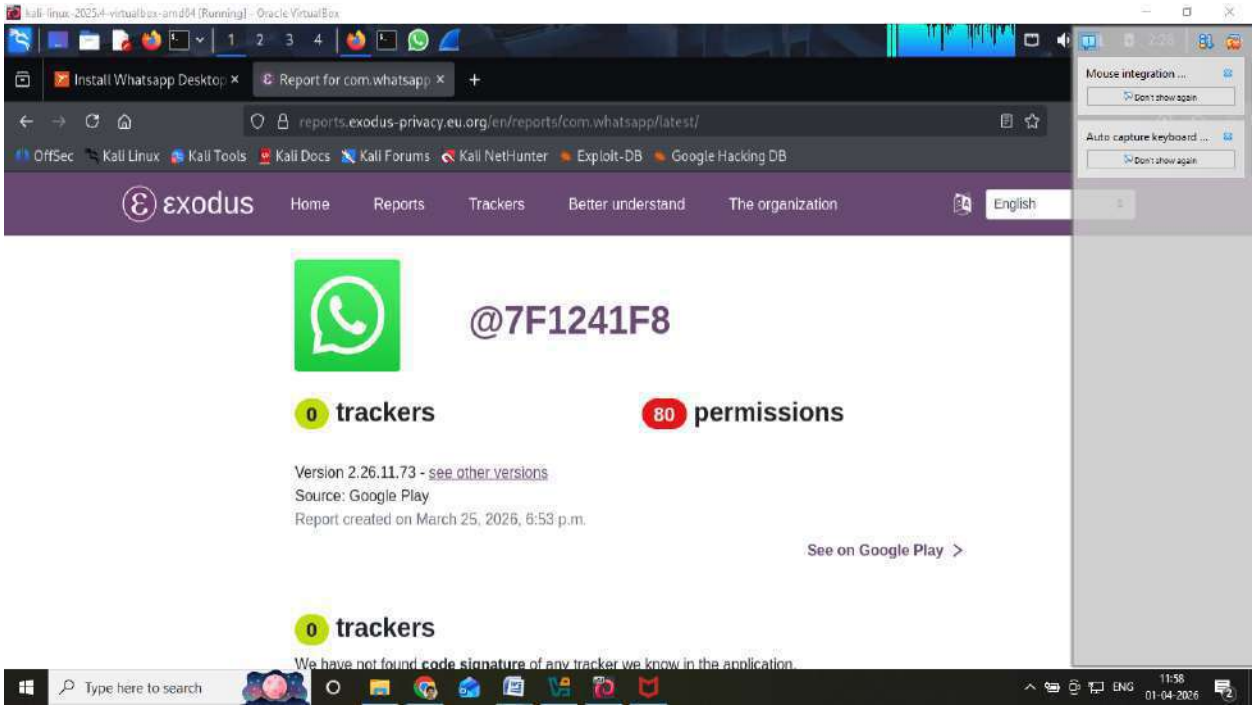
1. Open browser in Kali
2. Go to: https://reports.exodus-privacy.eu.org



3. Search for **WhatsApp**

Observation:

- Detects trackers like:
  - Google Analytics
  - Crashlytics
  - Facebook Analytics

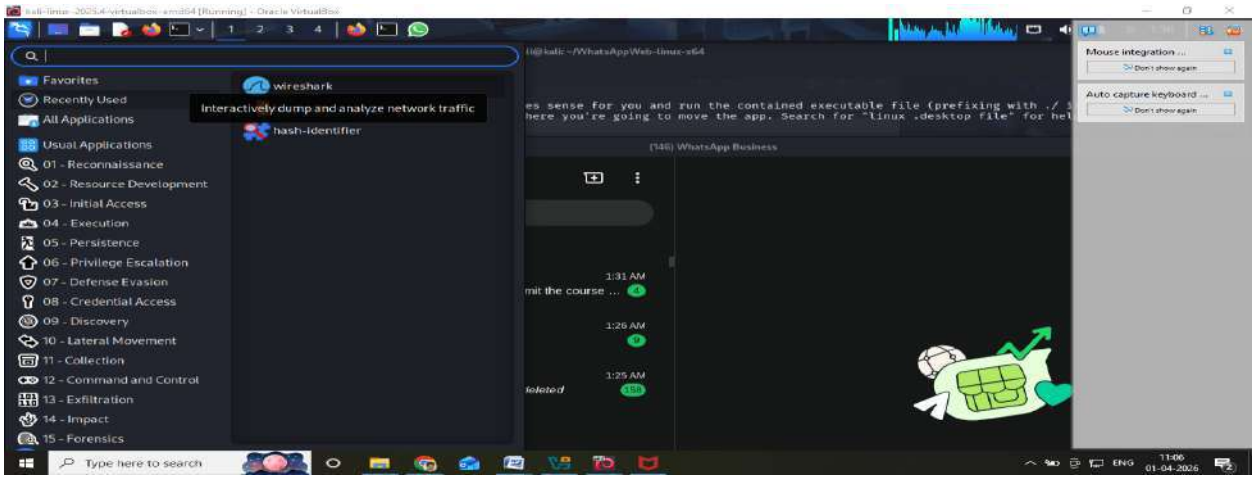


Record in lab:

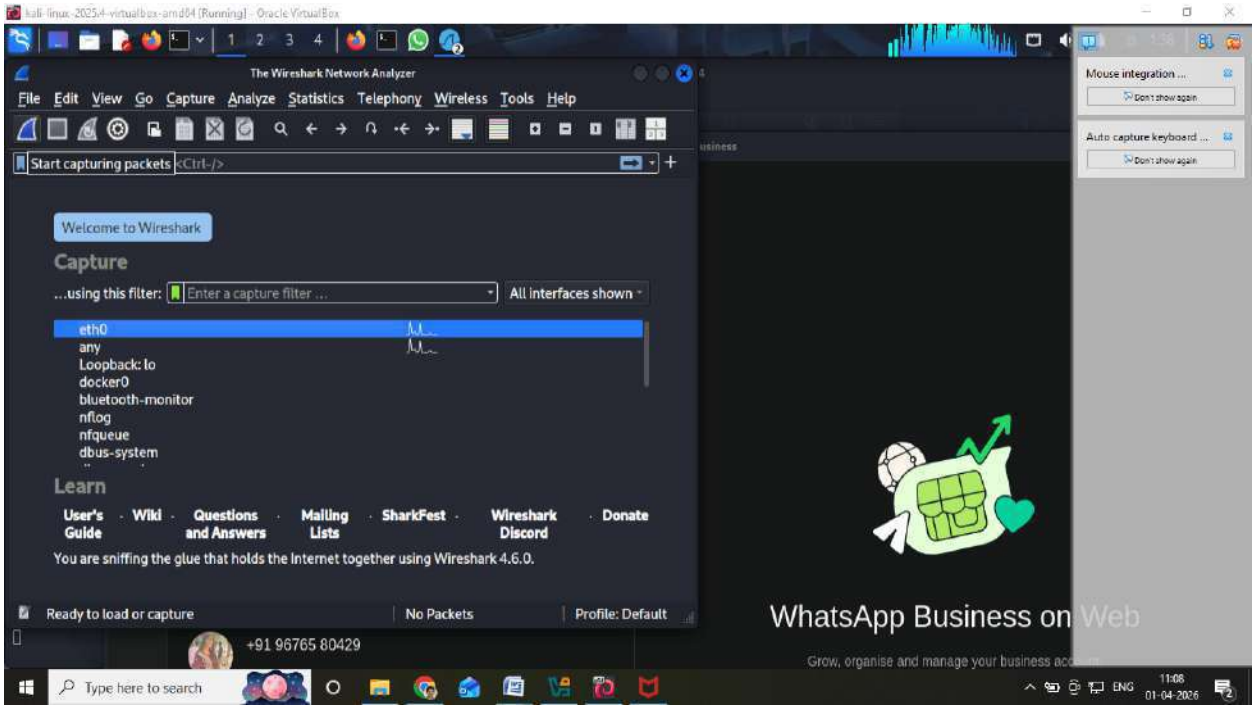
- Number of trackers
- Permissions used

Step 2: Start Wireshark

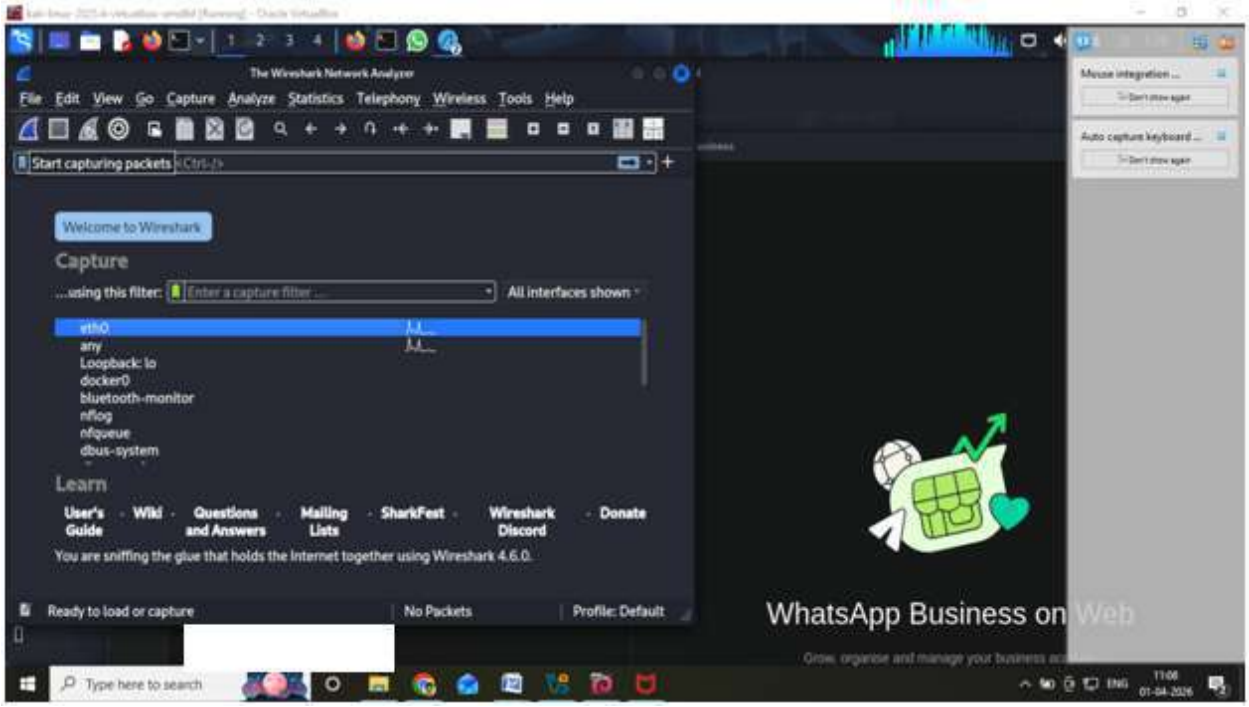
Wireshark



- Select interface (eth0 / wlan0)



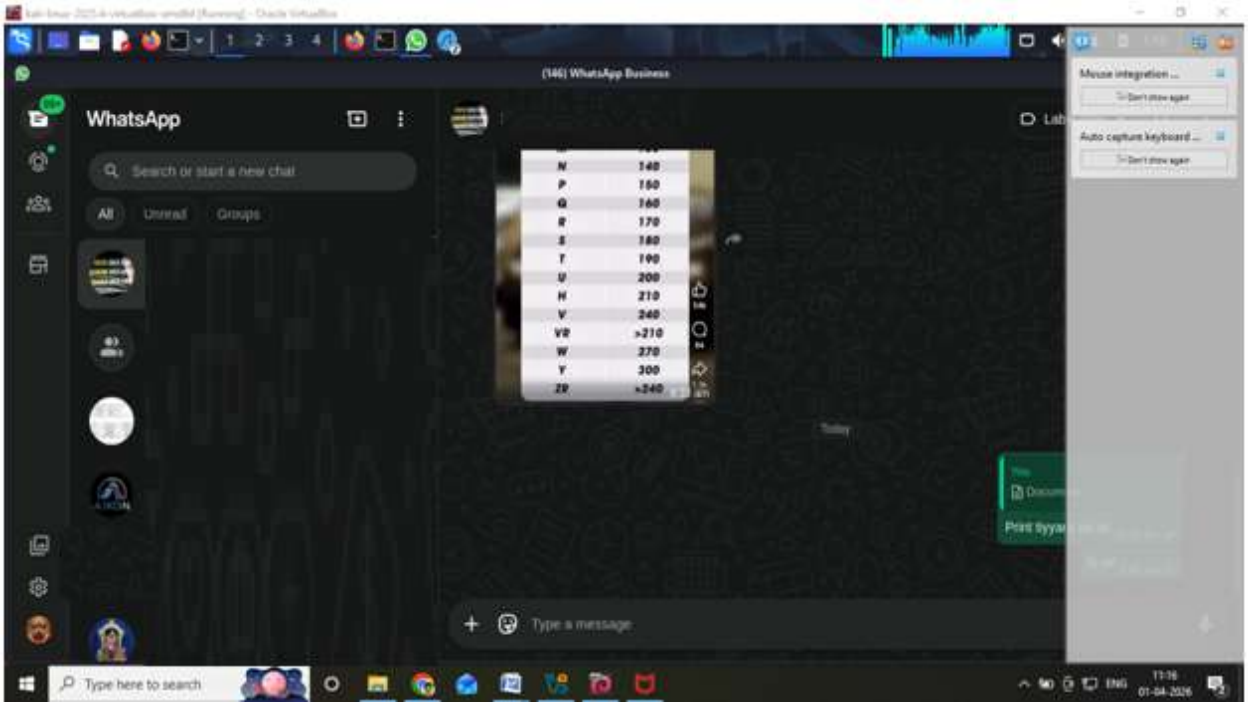
-
- Click **Start Capture**



Step 3: Generate Traffic

In WhatsApp Desktop:

- Send messages
- Send images
- Open chats

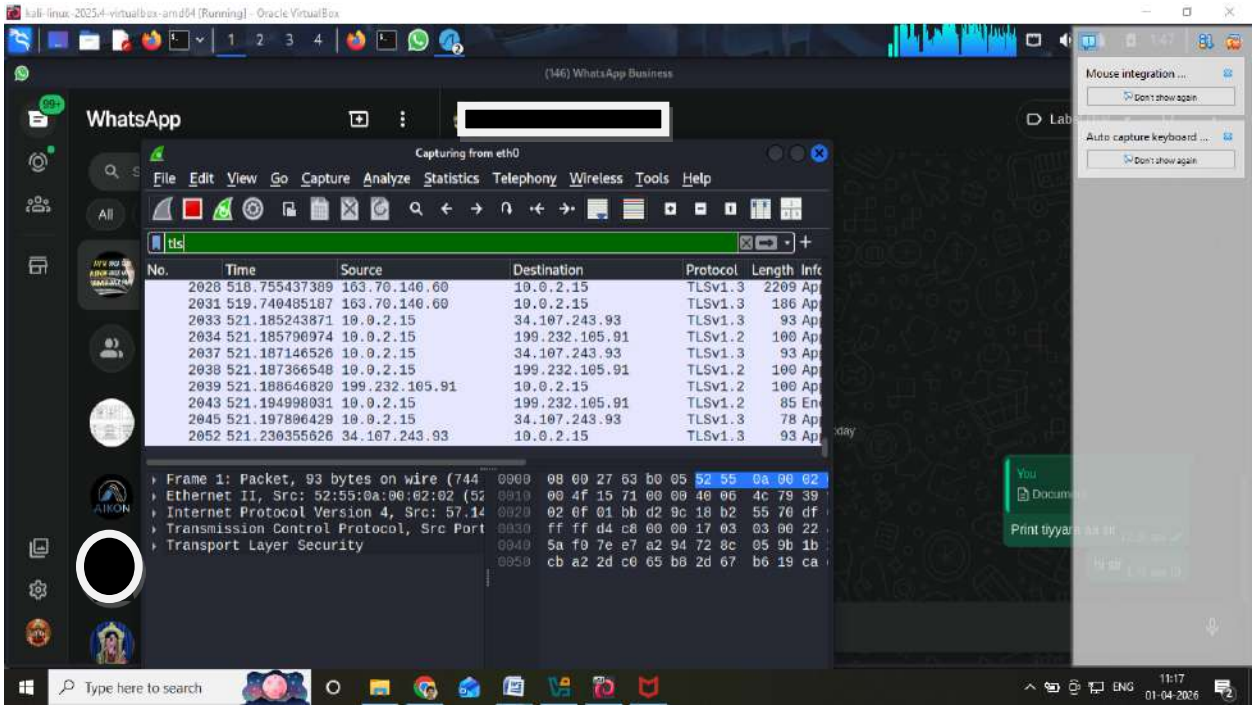


✅ **Step 4: Apply Filters**

Use filters:

tls

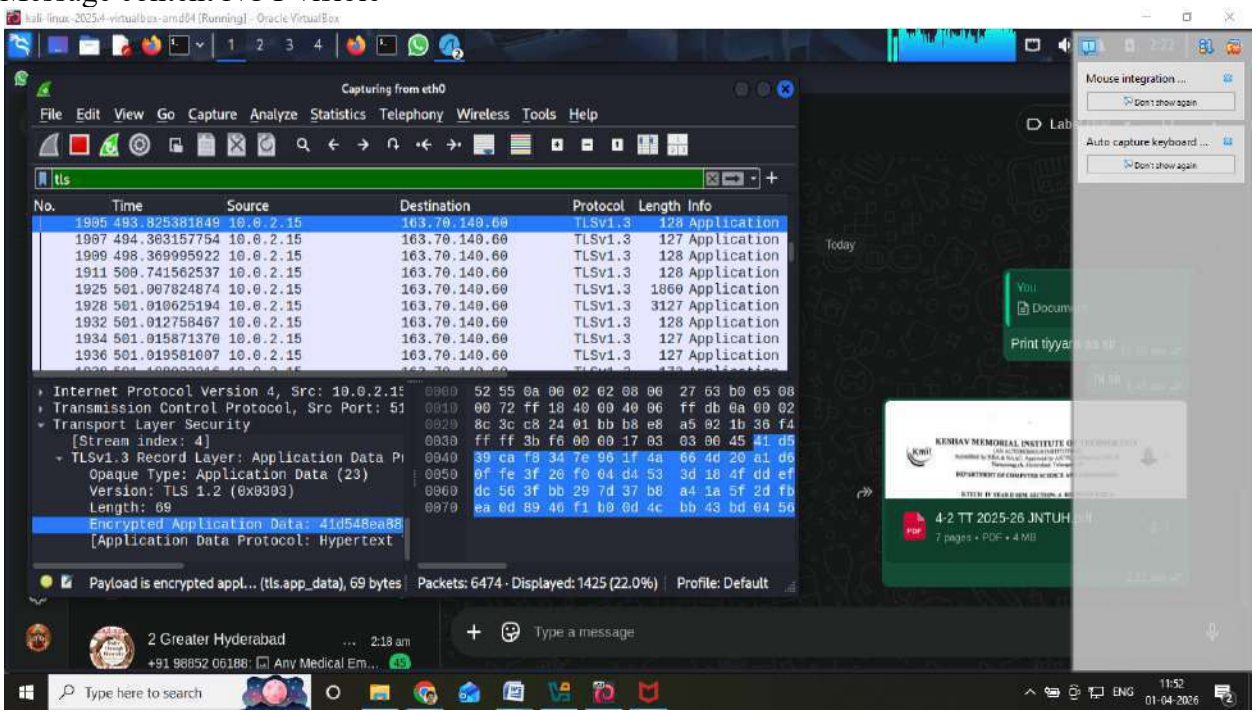
dns



Observations

- Encrypted traffic (TLS packets)
- Domains:
  - whatsapp.net
  - facebook.com

Message content NOT visible



Findings

End-to-End Encryption

Secure communication

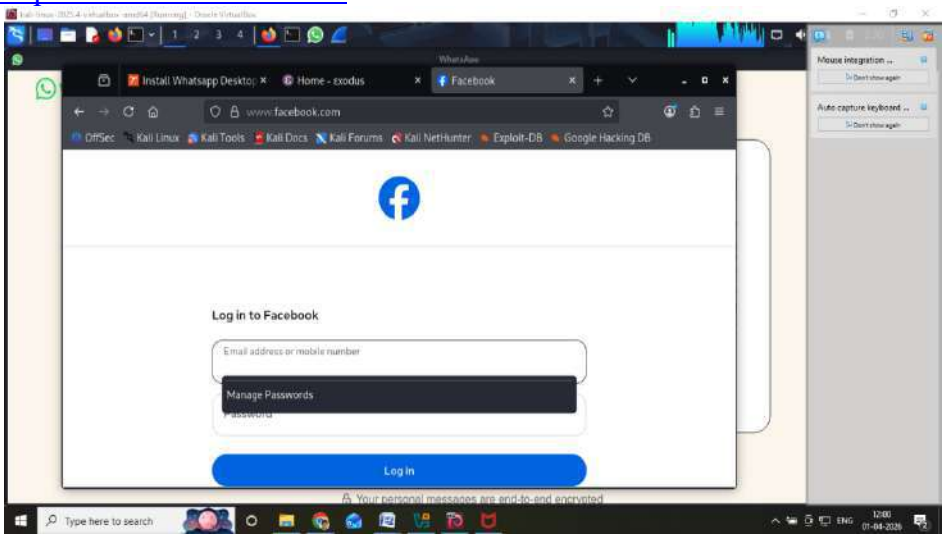
Metadata visible:

- IP address
- Time
- Server communication

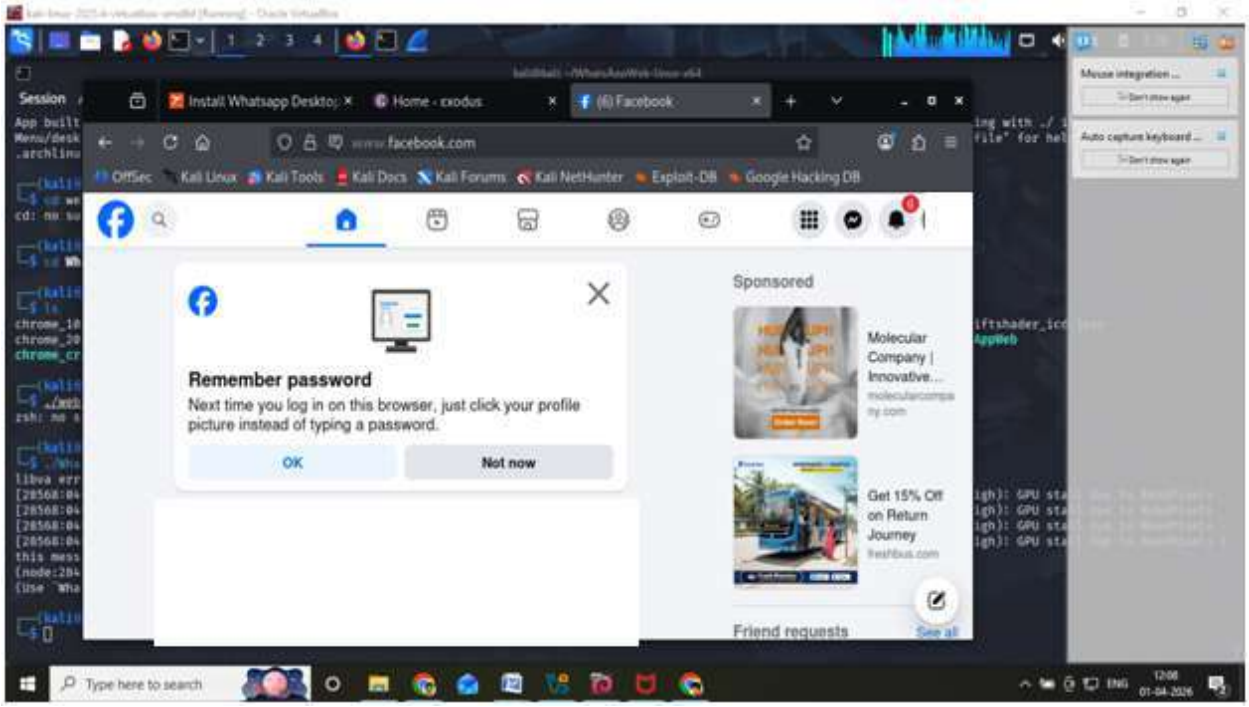
PART 2: Privacy Audit of Facebook Website

Step 1: Open Facebook

https://www.facebook.com

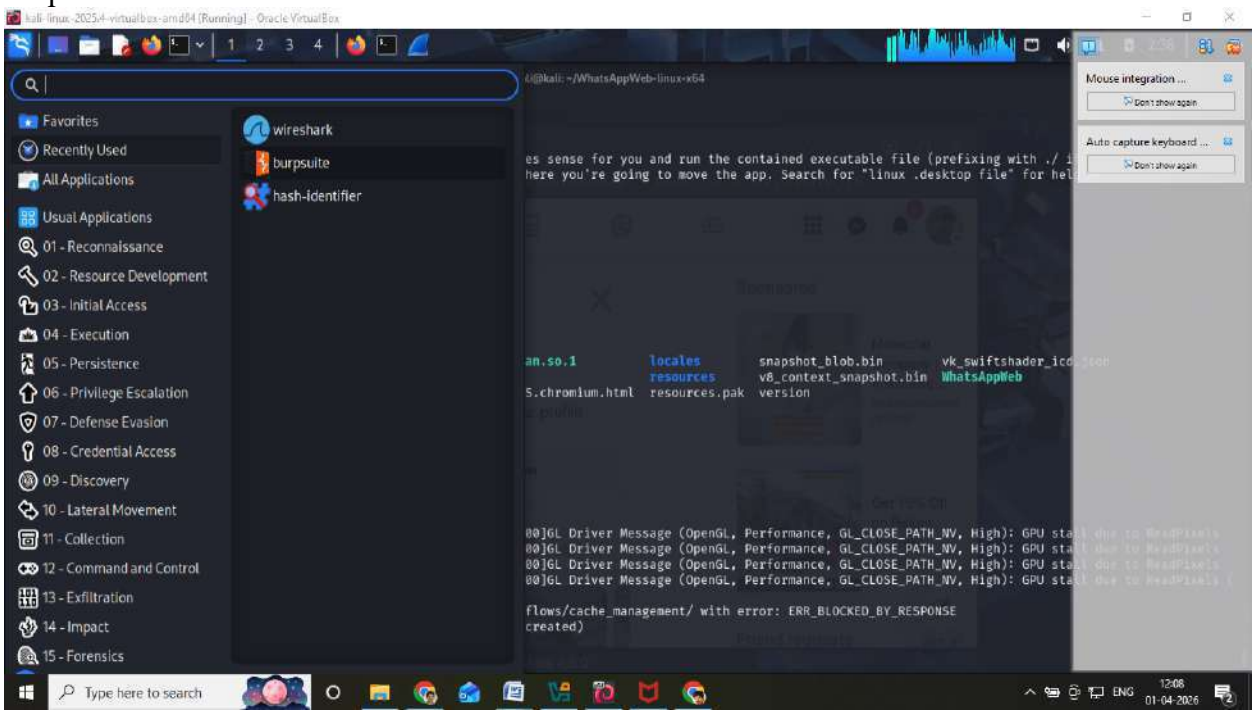


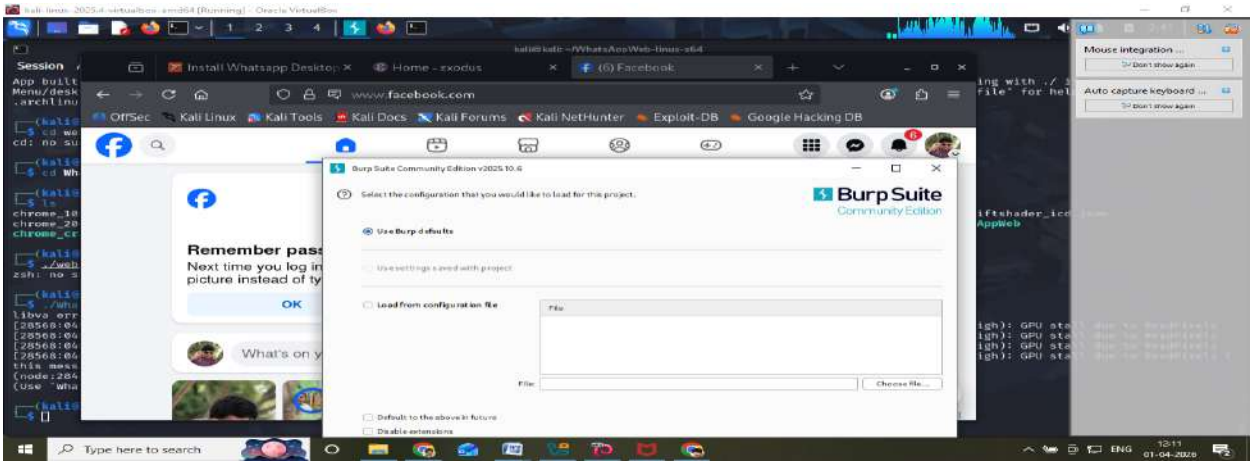
Login using test account



Step 2: Start Burp Suite

Burpsuite



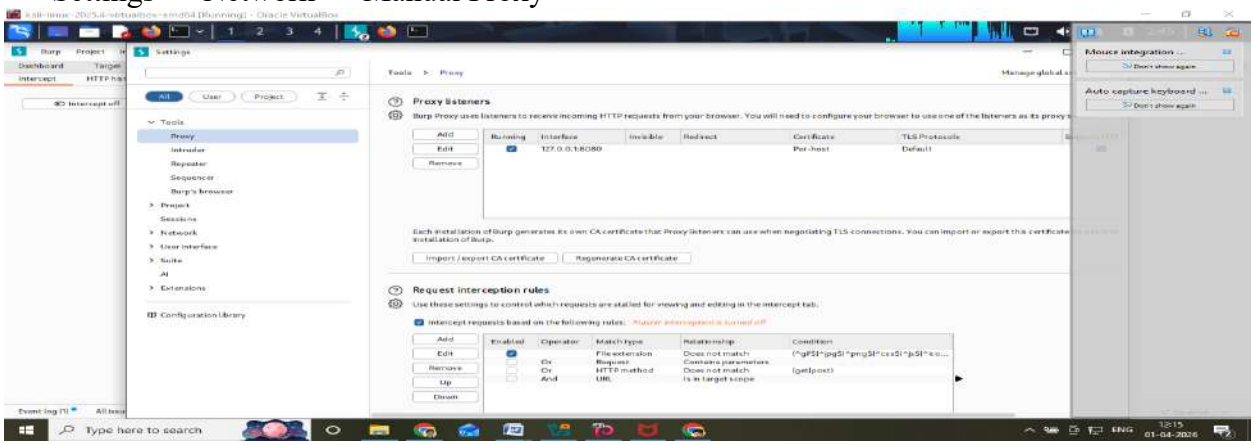


✅ Step 3: Configure Proxy

- IP: 127.0.0.1
- Port: 8080

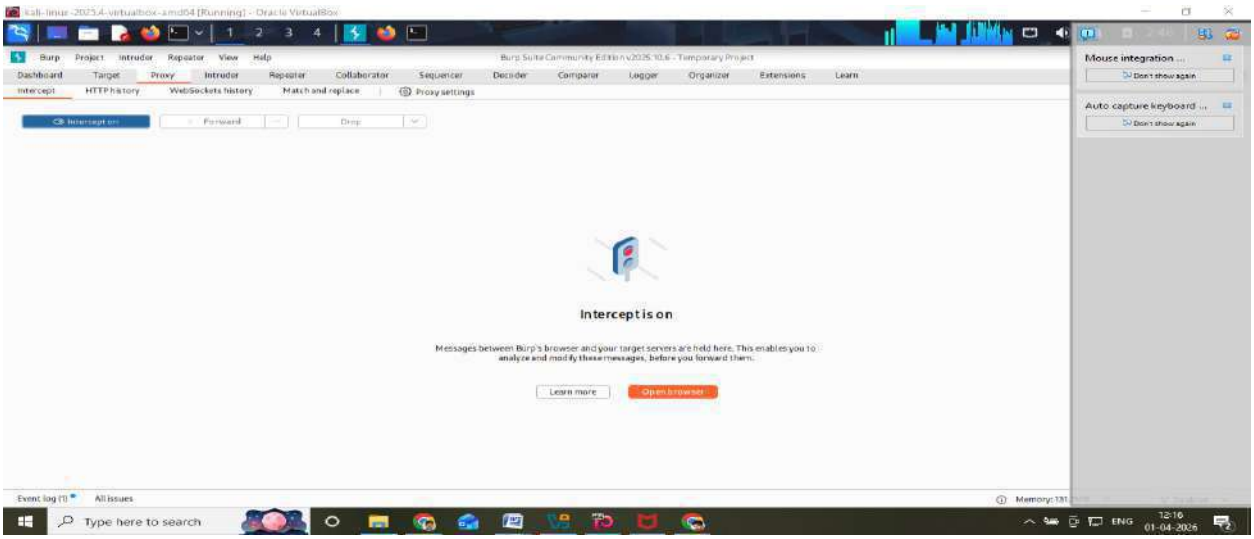
In browser:

- Settings → Network → Manual Proxy



✅ Step 4: Intercept Traffic

- Turn **Intercept ON**
- Reload Facebook



Observe:

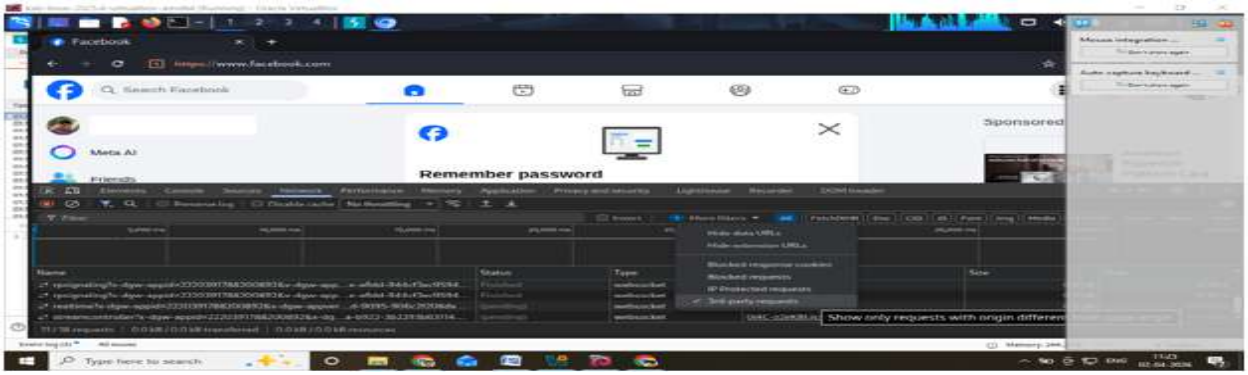
- Cookies (datr, fr)
- Session IDs
- Headers

Step 5: Analyze Tracking

Press **F12 → Network tab**

Look for:

- Ads
- Analytics scripts
- Third-party requests



Findings

Heavy tracking

Uses cookies for personalization

Third-party integrations

PART 3: Data Breach Case Study

Case Study 1: Facebook Data Breach

- 533 million users affected
- Data leaked:
    - Phone numbers
    - Emails

Cause:

- API vulnerability

Case Study 2: WhatsApp Pegasus Attack

- Spyware attack via missed call

- Remote device access

Step 3: Check Email Breach

Go to:

👉 https://haveibeenpwned.com

Enter email → check breach status

Analysis Points

- Type of attack
- Vulnerability
- Data exposed
- Prevention

Observations

- WhatsApp uses strong encryption
- Facebook performs extensive tracking
- Data breaches occur due to vulnerabilities

Result

Privacy audit was successfully performed using Kali Linux tools. WhatsApp ensures secure communication through encryption, while Facebook shows extensive tracking behavior. Data breach analysis highlighted risks of poor security practices.

Experiment 21 :Conducting a Security Audit and Risk Assessment

1. Aim

To perform a security audit on a Windows system and identify potential vulnerabilities by conducting a risk assessment and suggesting mitigation strategies.

2. Tools Required

- Windows Operating System (Windows 10/11)
- Windows Security (Defender / Antivirus)
- Command Prompt
- Task Manager
- Web Browser (Google Chrome / Microsoft Edge)

3. Introduction

A security audit is the process of evaluating a system's security measures to identify vulnerabilities. Risk assessment involves analyzing potential threats, vulnerabilities, and their impact on the system. It helps in implementing appropriate security controls to protect data and resources from cyber threats.

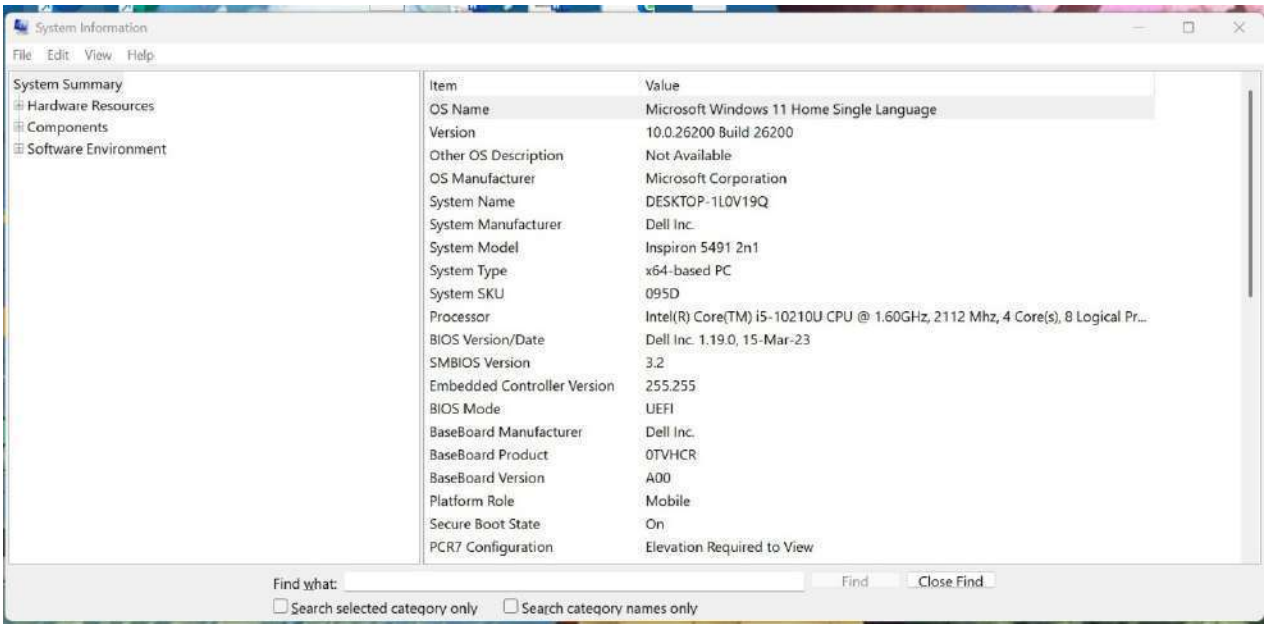
4. Procedure

STEP 1: System Information

- Press **Windows + R**
- Type: msinfo32

Note:

- OS Version
- System Type



Observation:

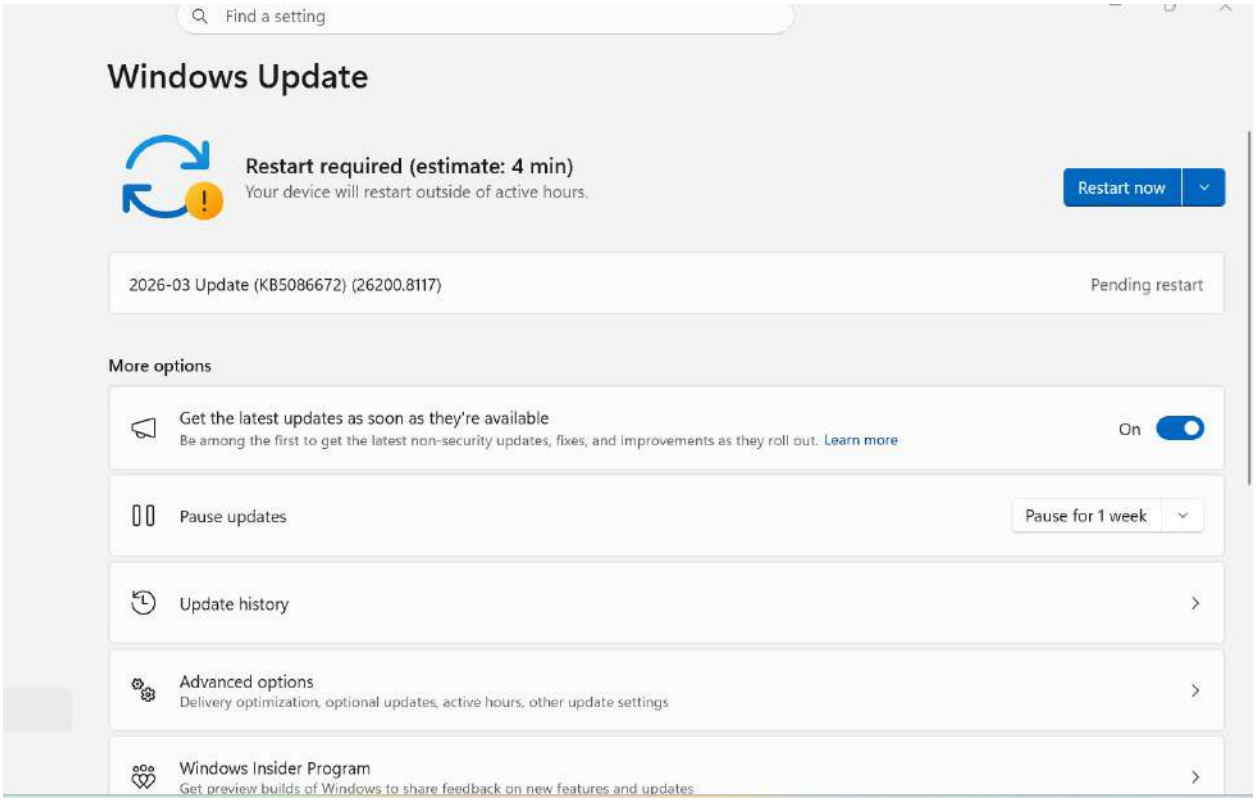
The system information was obtained using the *msinfo32* tool. The system is running on Microsoft Windows 11 Home Single Language, 64-bit operating system, with Intel i5 processor and UEFI-based architecture. Secure Boot is enabled, indicating a secure boot configuration. Secure Boot being enabled enhances system security by preventing unauthorized boot loaders.

STEP 2: Check

Windows

Updates Steps:

- Open **Settings**
- Go to **Windows Update**
- Click **Check for updates**



Observation:

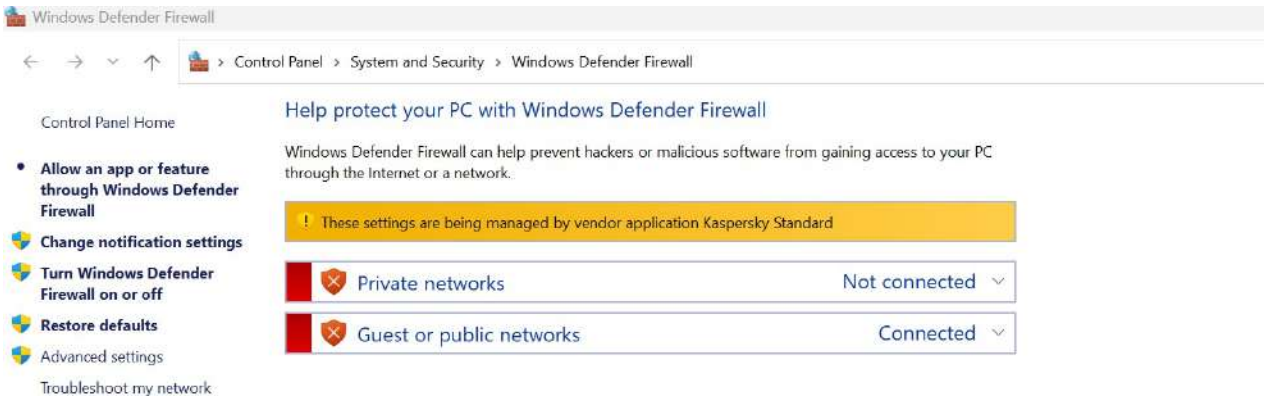
The Windows Update section was checked, and it was found that updates are available and pending installation. The system requires a restart to complete the update process.
Pending updates may expose the system to known vulnerabilities. Regular updates are essential to maintain system security and protect against threats.

STEP 3: Check

Firewall Status

Steps:

- Search: **Windows Defender Firewall**
- Click **Turn Windows Defender Firewall on or off**



Observation:
The firewall settings were checked using Windows Defender Firewall. It was observed that firewall settings are managed by Kaspersky antivirus, indicating that a third-party firewall is active on the system.
The presence of a third-party firewall ensures network protection; however, misconfiguration may still expose the system to risks.

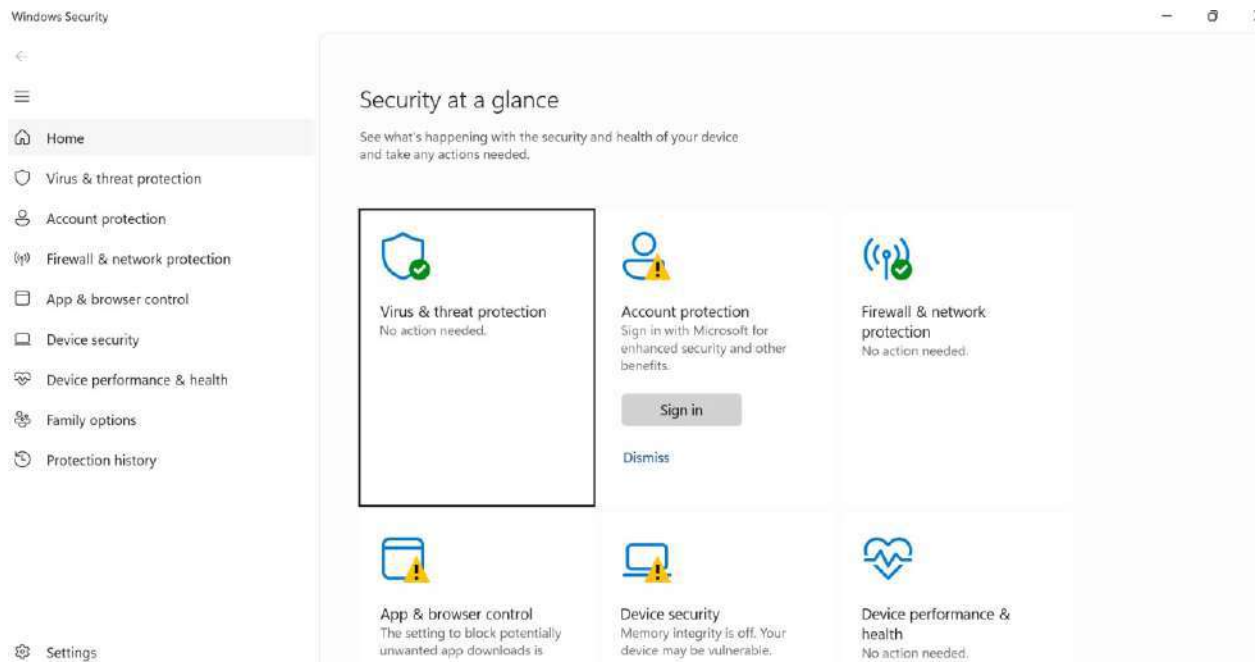
STEP 4: Check Antivirus

(Windows Security)

Steps:

- Open **Windows Security**
- Go to **Virus & Threat Protection**

Check:
- Real-time protection → ON
- Last scan → Recent



Observation:

The antivirus status was verified under Virus and Threat Protection. It was observed that Kaspersky antivirus is active, and no current threats were detected. Protection settings and updates are functioning properly. A third-party antivirus like Kaspersky provides advanced protection against malware, phishing, and other cyber threats.

STEP 5: Check Password

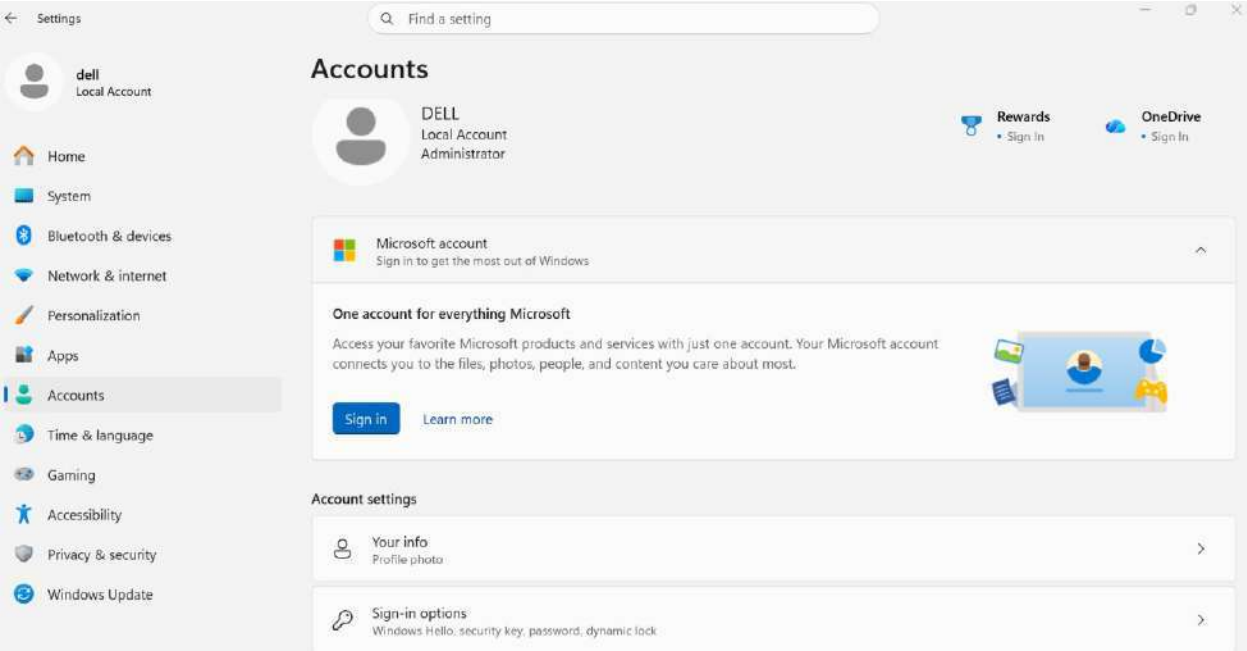
& Account Security

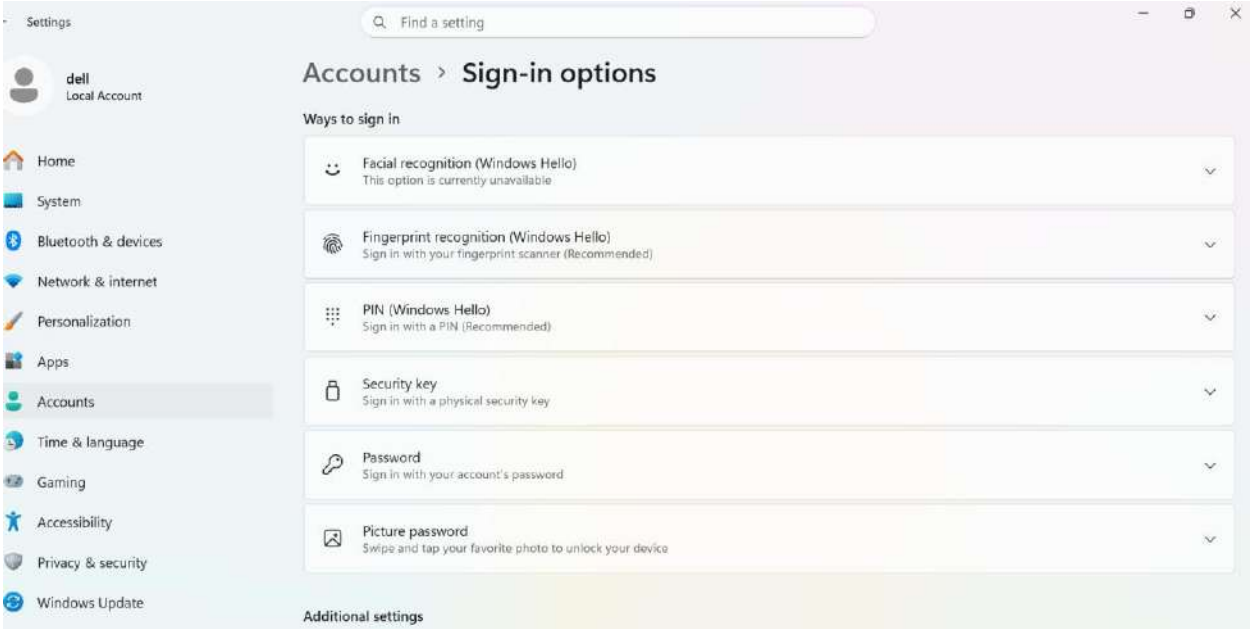
Steps:

- Settings → **Accounts → Sign-in options**

Check:

- Password / PIN enabled
- Windows Hello (optional)





Observation:

The account settings and sign-in options were checked. The system uses a local administrator account with multiple authentication methods such as password and PIN (Windows Hello), ensuring secure access.

Strong authentication mechanisms like PIN and biometric options enhance system security and prevent unauthorized access.

STEP 6: Check

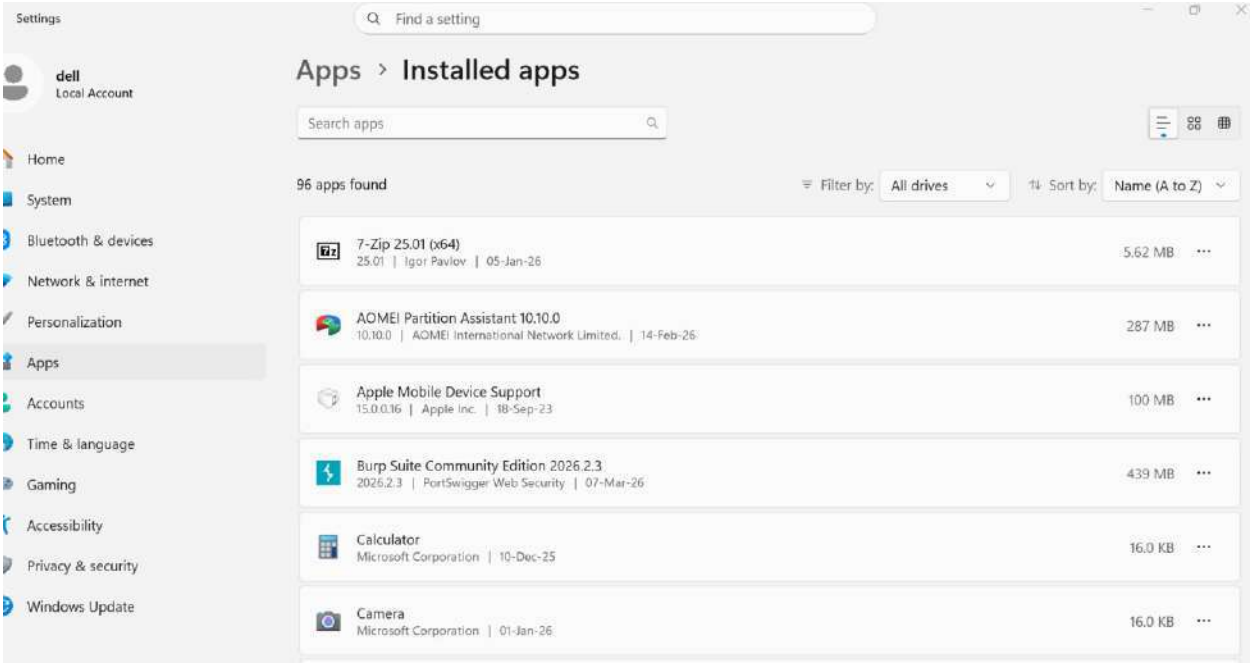
Installed

Applications Steps:

- Settings → **Apps** → **Installed apps**

Look for:

- Unknown software
- Cracked tools



Observation:

The list of installed applications was reviewed. No unauthorized or suspicious software was identified in the system.
Unverified or cracked software may introduce malware and compromise system security.

STEP 7: Check

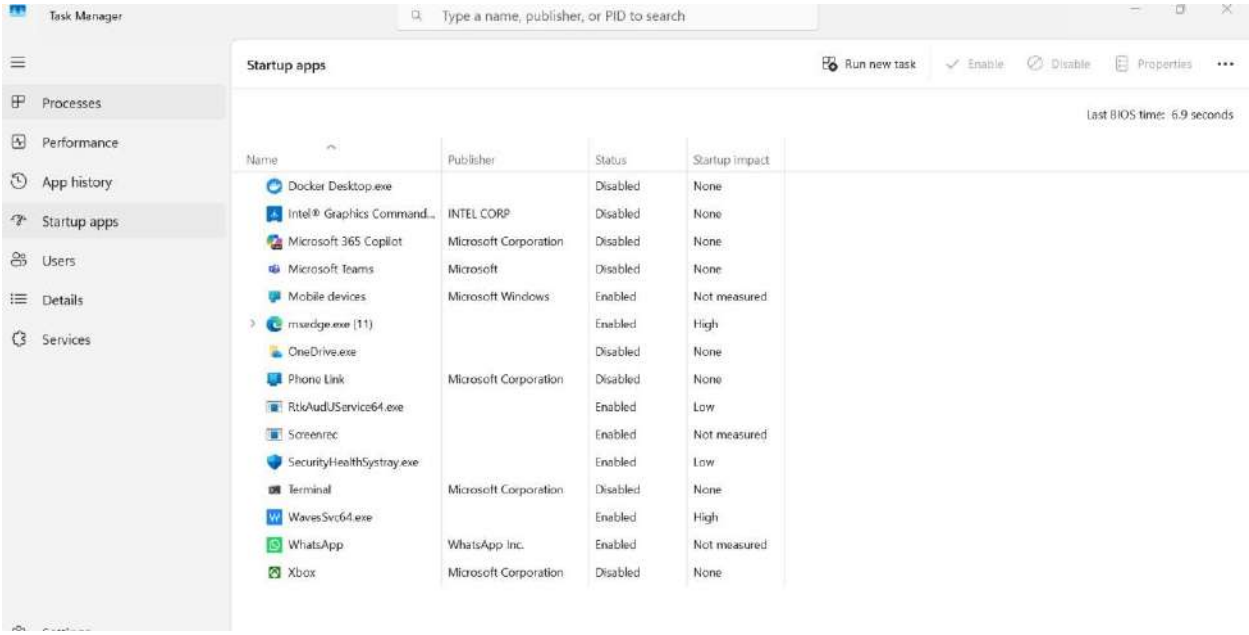
Startup

Programs Steps:

- Press **Ctrl + Shift + Esc**
- Go to **Startup tab**

Disable:

- Unknown apps



Observation:

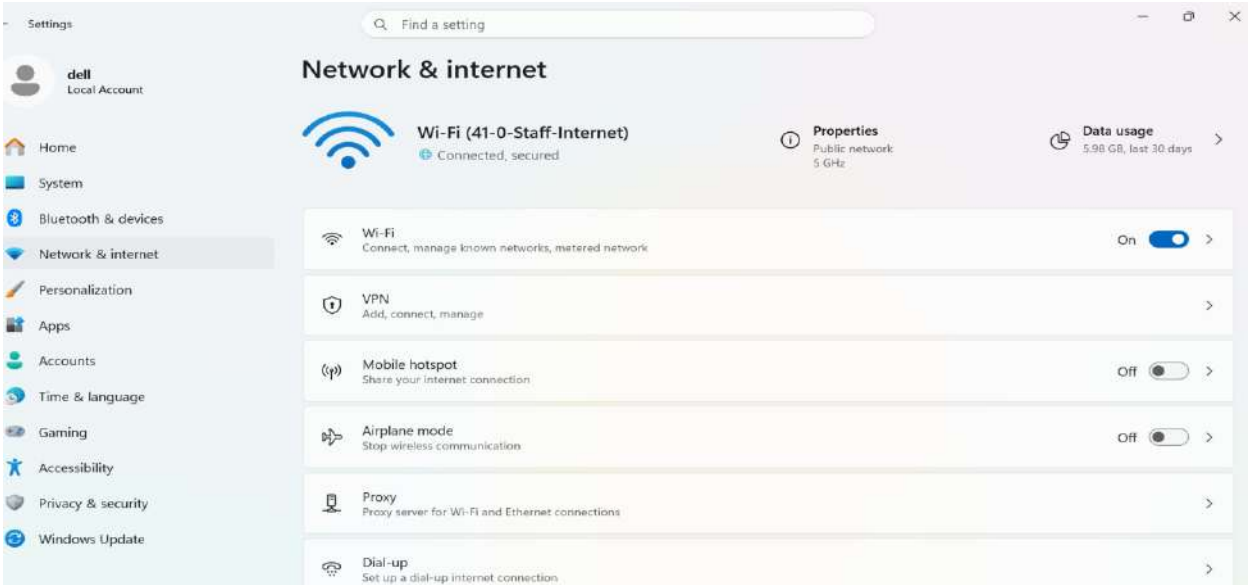
Startup applications were analyzed using Task Manager. Some applications were enabled at startup, which may impact system performance and security. Disabling unnecessary startup applications reduces attack surface and improves system performance.

STEP 8: Check

Network

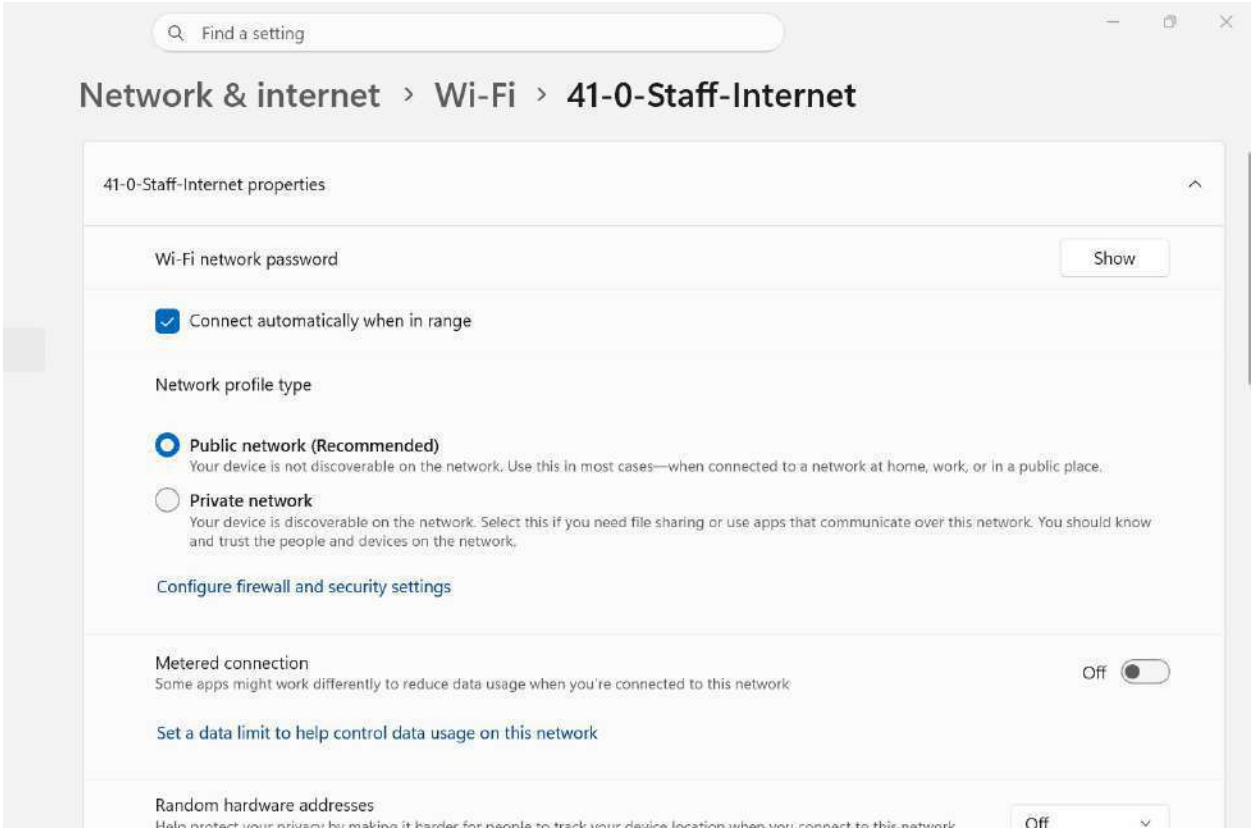
Security Steps:

- Settings → **Network & Internet**
- Check:
  - WiFi connection
  - Network type → Private/Public



Network Overview

- Wi-Fi connected → **41-0-Staff-Internet**
- Status → **Connected, secured**
- Shows general network info



Network Type

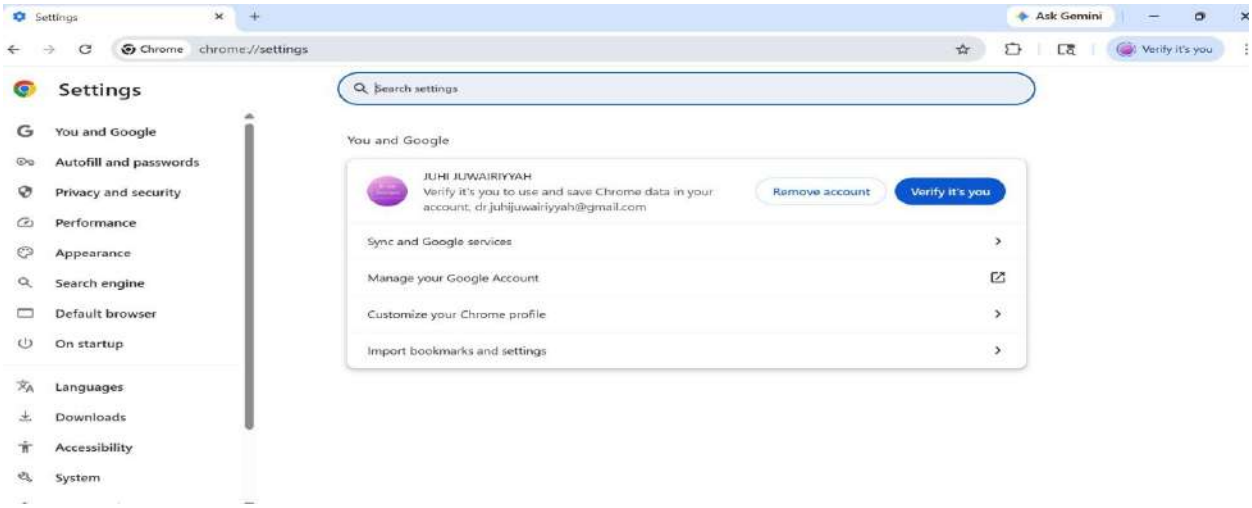
- Network profile → **Public network (selected)**
- Private option available

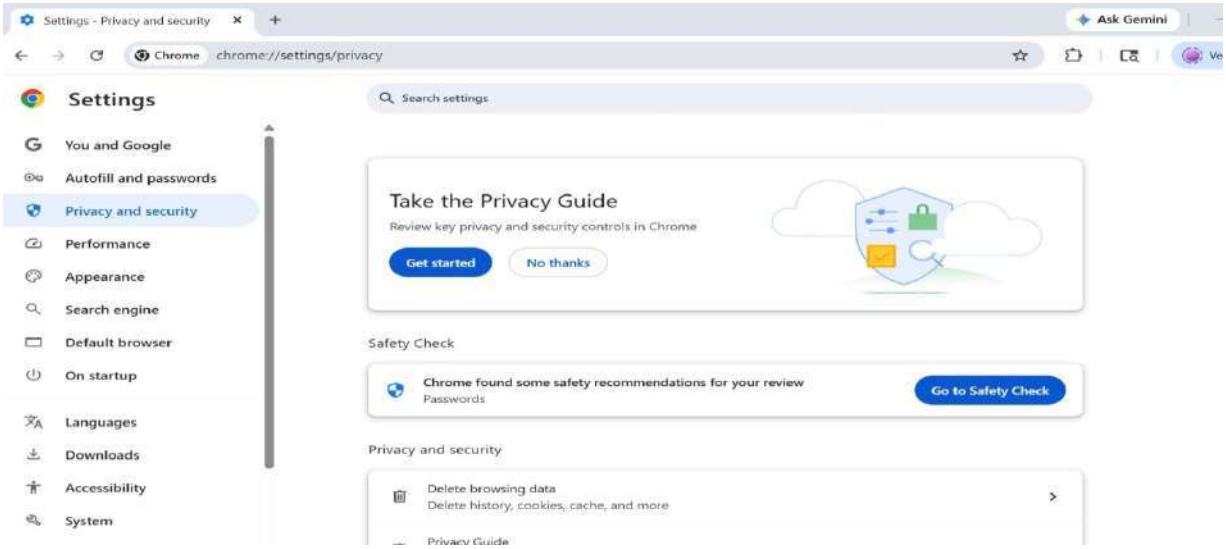
Observation:

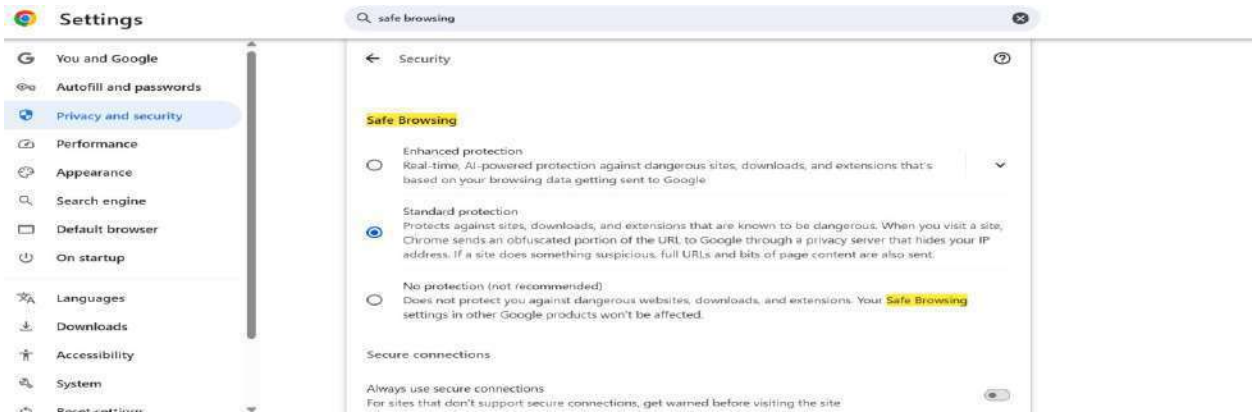
The network settings were analyzed, and the system is connected to a Wi-Fi network with a public network profile. This restricts device visibility and enhances security.
Using a public network profile improves security by limiting unauthorized access, especially in untrusted environments.

STEP 9: Check Browser Security Steps (Chrome/Edge):

- Go to **Settings → Privacy & Security**
- Check:
    - Safe browsing ON
    - Remove unknown extensions




Settings - Privacy and security
Ask Gemini
Chrome
chrome://settings/privacy
Settings
Search settings
You and Google
Autofill and passwords
Privacy and security
Performance
Appearance
Search engine
Default browser
On startup
Languages
Downloads
Accessibility
System
Take the Privacy Guide
Review key privacy and security controls in Chrome
Get started
No thanks
Safety Check
Chrome found some safety recommendations for your review
Passwords
Go to Safety Check
Privacy and security
Delete browsing data
Delete history, cookies, cache, and more



Observation:

The browser security settings were examined under the Privacy and Security section. Safe Browsing was found to be enabled with standard protection, ensuring protection against malicious websites and unsafe downloads.

Safe Browsing helps prevent phishing attacks, malware infections, and unsafe browsing activities.

STEP 10:

Check Data Backup Steps:

- Search: **Backup settings**
- Check:
  - OneDrive / External backup

: lab-user

Observation:

The backup settings were checked, and it was observed that OneDrive backup is not configured and system data is not being backed up.

Lack of proper backup mechanisms may lead to permanent data loss in case of system failure or cyber attacks such as ransomware.

STEP 11: Risk Assessment Table

Asset	Threat	Vulnerability	Risk Level	Solution
Personal Files	Data Loss	No Backup	High	Enable OneDrive
System	Malware	Unknown Apps	High	Remove apps
Network	Hacking	Public Network	Medium	Use Private
Account	Unauthorized Access	Weak Password	High	Strong password

STEP 12: Security Recommendations

✓ Technical Controls

- Enable firewall
- Enable antivirus
- Regular updates

Administrative Controls

- Strong passwords
- Avoid unknown downloads

User Awareness

- Avoid phishing links
- Safe browsing

STEP 13: Result

The system security audit identified potential vulnerabilities such as lack of backup and configuration risks. Suitable mitigation strategies were suggested to improve system security.